In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)
```

In [ ]:
```
!pip install pandas numpy scikit-learn lightgbm xgboost torch
```

In [ ]:
```
!pip install -U pip setuptools wheel
```

In [3]:
```
df = pd.read_csv(r"C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\Consolidated\oldunuseddata\mldataIn
dfq = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-dormancy\data\vw_co
dfqbulk = dfq[dfq.Stage==1]
dfqbulk = dfqbulk[['CT_Initial', 'CT_Initial_DT', 'WG_Initial_7Day_Cnt', 'WG_Initial_7Day_Cnt_DT', 'Bulk_Batch
```

```
C:\Users\EIJEL\AppData\Local\Temp\ipykernel_16188\3439832403.py:2: DtypeWarning: Columns (13,16,24,132,133,134,135,136,137,138) have
mixed types. Specify dtype option on import or set low_memory=False.
  dfq = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\data\vw_cotton_lineage_and_quality_june_fg_all_cols_lotcreateddt.csv')
```

In [3]:
```
dfqbulk.head()
```

Out [3]:

	CT_Initial	CT_Initial_DT	WG_Initial_7Day_Cnt	WG_Initial_7Day_Cnt_DT	Bulk_Batch
0	NaN	NaN	NaN	NaN	922
1	74.25	2018-02-02	92.25	2018-02-05	A77U
73	65.75	2018-02-14	88.00	2018-02-14	A77V
95	74.25	2018-01-05	92.40	2018-01-05	A77Y
405	37.25	2018-02-20	56.50	2018-01-25	A786

In [4]:
```
df_truck_with_pa_feature_id = pd.read_csv(r"C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\co
df_truck_with_pa_feature_id = df_truck_with_pa_feature_id.drop_duplicates(subset = ['pa_feature_id'])
cotton_with_truck = df.merge(df_truck_with_pa_feature_id[['pa_feature_id', 'bulk_batch', 'lb_received', 'inspc
```

In [9]:
```
df_truck_with_pa_feature_id.head()
```

Out [9]:

	season_yr	inspct_lot_nbr	receipt_dt	gin	carrier_driver	prcs_ord_nbr	bulk_batch	batch_nbr	load_nbr	variety	class	scale
0	2020	4.000349e+10	2021-01-14	RIVER BUSH FARMS	MALLETT & SONS TRUCK CO INCORPORATE	2502137892	E042	E042020977	7	DP2020B3XF	O	2020
1	2020	4.000349e+10	2021-01-13	Normark Farms LLC	MALLETT & SONS TRUCK CO INCORPORATE	2502137849	E046	E046020932	1	DP2020B3XF	B	2020
2	2020	4.000348e+10	2020-12-19	FARMERS GIN INCORPORATED	MALLETT & SONS TRUCK CO INCORPORATE	2502151190	E057	E057020804	4	DP2012B3XF	B	2020
3	2020	4.000342e+10	2020-10-16	MID-VALLEY COTTON GROWERS INC	AGRA TRADING LLC	2502138076	E073	E073002683	1	DP1646B2XF	O	2683
4	2020	4.000342e+10	2020-10-16	MID-VALLEY COTTON GROWERS INC	AGRA TRADING LLC	2502138076	E073	E073002685	3	DP1646B2XF	O	2685

In [5]:
```
# sorted(cotton_with_truck.columns)
```

In [6]:
```
# sorted(cotton_with_truck.columns)
```

In [7]:
```
# df_truck_with_pa_feature_id = \
# df_truck_with_pa_feature_id.sort_values(by = ['bulk_batch', 'lb_received'], \
#                                         ascending = [True, False]).drop_duplicates(subset = ['bulk_batch'], 
# df_single_batch = df.merge(df_truck_with_pa_feature_id[['pa_feature_id', 'bulk_batch']], on='pa_feature_id')
```

In [5]:
```
dffuzzyquery = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-dormancy\da
dffuzzyquery = dffuzzyquery[dffuzzyquery['CT_Current_DT'].notna()]
```

```
C:\Users\EIJEL\AppData\Local\Temp\ipykernel_16188\579294774.py:1: DtypeWarning: Columns (35,81,137,139) have mixed types. Specify dtype
option on import or set low_memory=False.
  dffuzzyquery = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\data\truck_to_quality\fuzzy_lineage_query.csv')
```

In [9]:
```
# sorted(dffuzzyquery.columns)
```

In [10]:
```
dffuzzyquery.head()
```

Out [10]:

	SEASON_YR	GRWNG_CNTRCT	PURCH_ORD_NBR	FIELD	GRWR_PYMNT	FUZZY_PRCS_ORD_NBR	ORD_TYP_CD	agreement_typ_cd	LOAD_MTRL_N
3	2020	4600000072	613322461.0	1646-O	627875.91	2502130823	ZG20	ZTT2	12269510
7	2020	4600000060	613300400.0	1646-O	399918.64	2502133098	ZG20	ZTT2	12269510
8	2020	4600000060	613300400.0	1820-O	399918.64	2502133099	ZG20	ZTT2	12528885
12	2020	4608651771	613315791.0	1522-F	934190.50	2502138421	ZG21	ZTA2	12206829
14	2020	4600000010	613322447.0	1822-O	275838.20	2502130272	ZG20	ZTT2	12528569

In [6]:
```
df_final = dffuzzyquery.merge(cotton_with_truck, left_on='FUZZY_INSPCT_LOT_NBR', right_on='inspct_lot_nbr')
```

In [7]:
```
df_final.shape
```

Out [7]: (922, 251)

In [13]:
```
df_final.SEASON_YR.value_counts()
```

Out [13]:
```
SEASON_YR
2023    272
2020    200
2024    190
2021    143
2022    117
Name: count, dtype: int64
```

In [8]:
```
df_single_batch_latest = df_final.copy()
```

In [17]:
```
############################################ ignore above code ############################################
```

In [9]:
```
df_single_batch_latest.harvest_date = pd.to_datetime(df_single_batch_latest.harvest_date)
df_single_batch_latest.CT_Current_DT = pd.to_datetime(df_single_batch_latest.CT_Current_DT)
df_single_batch_latest.WG_Current_DT = pd.to_datetime(df_single_batch_latest.WG_Current_DT)
```

In [10]:
```
df_single_batch_latest['harvest_to_CT_dt'] = df_single_batch_latest.CT_Current_DT - df_single_batch_latest.harv
```

In [11]:
```
df_single_batch_latest['harvest_to_WG_dt'] = df_single_batch_latest.WG_Current_DT - df_single_batch_latest.harv
```

In [12]:
```
def convert_days_to_int(series):
    """Convert 'n days' strings to integers (e.g., '5 days' -> 5).
    Handles pandas.NA for missing integer values.
    """
    if not isinstance(series, pd.Series):
        return series # Return as is if not a Series

    # Apply only if the series contains strings
    if series.dtype == 'object' or pd.api.types.is_string_dtype(series):
        return pd.to_numeric(series.str.replace(' days', '', regex=False), errors='coerce')
    if pd.api.types.is_timedelta64_dtype(series.dtype):
        return series.map(lambda x: x.days).astype(np.int32)
    return series # Return original series if not string type

df_single_batch_latest.defol_to_harvest = convert_days_to_int(df_single_batch_latest.defol_to_harvest)
# df_single_batch_latest.harvest_to_CT_dt = convert_days_to_int(df_single_batch_latest.harvest_to_CT_dt)
# df_single_batch_latest.harvest_to_WG_dt = convert_days_to_int(df_single_batch_latest.harvest_to_WG_dt)
```

In [13]:
```
df_single_batch_latest.defol_to_harvest.dtype
```

Out [13]:
```
dtype('float64')
```

In [ ]:
```
df_single_batch_latest.to_csv('field_to_facility.csv', index=False)
```

In [21]:
```
df_single_batch_latest.SEASON_YR.value_counts()
```

Out [21]:
```
SEASON_YR
2023    272
2020    200
2024    190
2021    143
2022    117
Name: count, dtype: int64
```

In [15]:
```
# df_single_batch = df_single_batch_prev_with_20232024
df_single_batch = df_single_batch_latest.copy()
```

In [19]:
```
sorted(df_single_batch.columns)
```

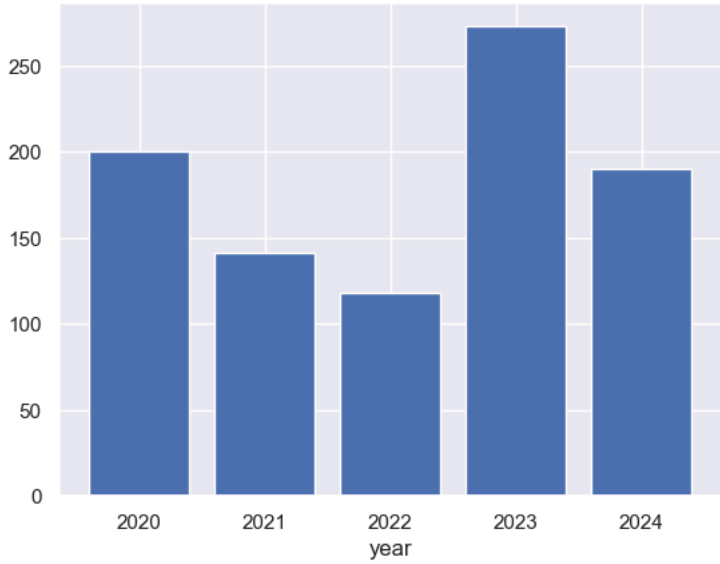
Out [19]:
```
['ABA_Level',
 'Actual_Seed_Per_LB',
 'Appearance_Rating',
 'BLACK_ORD_TYP_CD',
 'BLACK_PLNT_CD',
 'BLACK_PRCS_ORD_NBR',
 'Bulk_Batch',
 'CT_Current',
 'CT_Current_DT',
 'CT_Date_Abn',
 'CT_Date_Dead',
 'CT_Date_FH',
 'CT_Date_Resample',
 'CT_Date_Resample_Abn',
 'CT_Date_Resample_DT',
 'CT_Date_Resample_Dead',
 'CT_Date_Resample_FH',
 'CT_Date_Resample_SS',
 'CT_Date_SS',
 'CT_Initial_Abn',
 'CT_Initial_DT',
 'CT_Initial_Dead',
 'CT_Initial_FH',
 'CT_Initial_Resample',
 'CT_Initial_Resample_Dead',
 'CT_Initial_SS',
 'CT_Initial_x',
 'CT_Initial_y',
 'CT_Test',
 'CT_Test_DT',
 'CT_Testing_Complete',
 'Cleanout_PCT',
 'Clothianidin',
 'Cry1Ac_Bollgard_Strip_Test',
 'Cry2Ab_Bollgard_Strip_Test',
 'Cut_Test_Discolored',
 'Cut_Test_Immature',
 'DAY4_CNT_IN_WG1',
 'DAY4_CNT_IN_WG2',
 'DAY7_CNT_IN_WG3',
 'DAY7_CNT_IN_WG3_DT',
 'Delintability',
 'Dicamba_Lateral_Strip_PCT',
 'Dust_Off',
 'External_2_4_D_Strip_Test',
 'External_Cry1Ac_Strip_Test',
 'External_Cry1F_Strip_Test',
 'External_Production_IL',
 'External_RR_Strip_Test',
 'External_VIP3A_Strip_Test',
 'FFA',
 'FIELD',
 'FUZZY_BATCH_NBR',
 'FUZZY_BRND_DESC',
 'FUZZY_CROP_CD',
 'FUZZY_GI_QTY',
 'FUZZY_GI_QTY_SSU',
 'FUZZY_GR_QTY',
 'FUZZY_GR_QTY_SSU',
 'FUZZY_INSPCT_LOT_NBR',
 'FUZZY_MTRL_DESC',
 'FUZZY_MTRL_GRP_NM',
 'FUZZY_MTRL_NBR',
 'FUZZY_MTRL_TYP_CD',
 'FUZZY_PKG_TYP_DESC',
 'FUZZY_PLNT_CD',
 'FUZZY_PRCS_ORD_NBR_x',
 'FUZZY_PRCS_ORD_NBR_y',
 'FUZZY_TRAIT',
 'FUZZY_VRTY_DESC',
 'Fluxapyroxad',
 'GRWNG_CNTRCT',
 'GRWR_PYMNT',
 'Heavy_Mechanical_Damage',
 'Imidacloprid',
 'Inert_Matter',
 'LOAD_BATCH_NBR',
 'LOAD_GI_QTY',
 'LOAD_GI_QTY_SSU',
 'LOAD_GR_QTY',
 'LOAD_GR_QTY_SSU',
 'LOAD_MTRL_NBR',
```

```
'LOAD_PLNT_CD',
'LOT_CREATED_DT',
'Liberty_Lateral_Strip_PCT',
'Light_Mechanical_Damage',
'Lint_Loss_PCT',
'Lygus_PCT',
'Mechanical_Damage',
'Medium_Mechanical_Damage',
'Metalaxyl',
'Moisture',
'Myclobutanil',
'ORD_TYP_CD',
'ORIGIN_REGION',
'PA_ID',
'PURCH_ORD_NBR',
'Pure_Seed',
'Pyraclostrobin',
'QM_UD_CD',
'RR_Lateral_Strip_PCT',
'SEASON_YR',
'Seed_Temperature',
'State_Lab_Cool',
'State_Lab_Final_Germ',
'State_Lab_Final_Germ_DT',
'Tetrazolium_High_Vigor',
'Tetrazolium_Predicted_Germ',
'USBL_UNITS_SSU',
'USG_DECSN_INSPCT_CD',
'VIP3A_Bollgard_Strip_Test',
'Vigor_ABN_121',
'Vigor_FH_123',
'Vigor_SS_122',
'WG3_Date_ABN',
'WG3_Date_ABN_DT',
'WG3_Date_DEAD',
'WG3_Date_DEAD_DT',
'WG3_Date_FH',
'WG3_Date_FH_DT',
'WG_Current',
'WG_Current_DT',
'WG_Date_7Day_Cnt',
'WG_Date_7Day_Cnt_DT',
'WG_Date_Abn',
'WG_Date_Dead',
'WG_Date_FH',
'WG_Date_Resample_7Day_Cnt',
'WG_Date_Resample_7Day_Cnt_DT',
'WG_Date_Resample_Abn',
'WG_Date_Resample_Dead',
'WG_Date_Resample_FH',
'WG_Date_Resample_SS',
'WG_Date_SS',
'WG_Initial_7Day_Cnt_DT',
'WG_Initial_7Day_Cnt_x',
'WG_Initial_7Day_Cnt_y',
'WG_Initial_Abn',
'WG_Initial_Dead',
'WG_Initial_FH',
'WG_Initial_Resample_7Day_Cnt',
'WG_Initial_Resample_7Day_Cnt_DT',
'WG_Initial_Resample_Abn',
'WG_Initial_Resample_Dead',
'WG_Initial_Resample_FH',
'WG_Initial_Resample_SS',
'WG_Initial_SS',
'WG_Testing_Complete',
'Xdamage',
'Xmaturity',
'Xtexture',
'accepted_modules',
'agreement_typ_cd',
'ans_variety',
'bales_per_module',
'bolls_open_1',
'bolls_open_1_date',
'bolls_open_2',
'bolls_open_2_date',
'bolls_open_3',
'bolls_open_3_date',
'bolls_open_4',
'bolls_open_4_date',
'bulk_batch',
'catch_rate',
'centroid',
'comments',
'cotton_maczone',
'country',
'county',
'crop',
'cumulated_dd60',
'cumulated_soil_moisture',
'date',
'dd60',
'defol_to_harvest',
'defol_to_harvest_average_dd60',
'defol_to_harvest_average_heat',
'defol_to_harvest_cum_dd60',
'defol_to_harvest_cum_heat',
'defoliation_1',
'defoliation_2',
'defoliation_3',
'defoliation_date',
'entityid',
'field_status',
'fuzzy_estimate',
'grid_id',
'group_id',
'harvest_date',
'harvest_date_fill',
'harvest_to_CT_dt',
'harvest_to_WG_dt',
'inspct_lot_nbr',
'irrigation',
```

```
 'irrigation_type',
 'lat',
 'lb_received',
 'lon',
 'maczone',
 'new_variety',
 'nodes_above_cracked_boll_1',
 'nodes_above_cracked_boll_1_date',
 'nodes_above_cracked_boll_2',
 'nodes_above_cracked_boll_2_date',
 'nodes_above_white_flower_1',
 'nodes_above_white_flower_1_date',
 'nodes_above_white_flower_2',
 'nodes_above_white_flower_2_date',
 'nodes_above_white_flower_3',
 'nodes_above_white_flower_3_date',
 'pH',
 'pa_area',
 'pa_feature_id',
 'pa_wkt',
 'pa_year',
 'pf_feature_id',
 'pf_name',
 'pf_type',
 'pf_wkt',
 'pf_year',
 'plant',
 'planted_surface',
 'planting_area_id',
 'planting_date',
 'plantname',
 'pp_day10_avg_dd60',
 'pp_day10_avg_soilmoisture',
 'pp_day10_cum_dd60',
 'pp_day10_cum_soilmoisture',
 'pp_day5_avg_dd60',
 'pp_day5_avg_soilmoisture',
 'pp_day5_cum_dd60',
 'pp_day5_cum_soilmoisture',
 'questiontext',
 'random_days',
 'random_days_high',
 'random_days_low',
 'rejected_modules',
 'rm',
 'row_number',
 'row_spacing',
 'season',
 'season_length',
 'seeding_rate',
 'spcl_trtmnt_cd',
 'state',
 'thryvon',
 'total_modules',
 'var_bin',
 'variety',
 'yield',
 'zipcode']
```

In [27]:

```python
# df_single_batch.date
import seaborn as sns
sns.set_theme()
df_single_batch['date'] = pd.to_datetime(df_single_batch['date'])

# Extract the year and get the value counts
year_counts = df_single_batch['date'].dt.year.value_counts()
df_single_batch['year'] = df_single_batch['date'].dt.year
# print(year_counts)
plt.bar(year_counts.index,year_counts)
plt.xlabel('year')
plt.xticks([2020, 2021, 2022, 2023, 2024])
# plt.show()
# plt.yscale('log')
plt.savefig('year_dist.png', dpi = 600)
```



In [21]:
```
df_single_batch.to_csv('train_data.csv', index=False)
```

In [17]:
```
# lets use all cols
cols_to_train = [
 'irrigation_type',
 'rm',
 'cumulated_dd60',
 'cumulated_soil_moisture',
 'dd60',
 'defol_to_harvest',
 'defol_to_harvest_average_dd60',
 'defol_to_harvest_average_heat',
 'defol_to_harvest_cum_dd60',
 'defol_to_harvest_cum_heat',
    # 'harvest_to_CT_dt',
    # 'harvest_to_WG_dt',
    "maczone",
  'pp_day10_avg_dd60',
 'pp_day10_avg_soilmoisture',
 'pp_day10_cum_dd60',
 'pp_day10_cum_soilmoisture',
 'pp_day5_avg_dd60',
 'pp_day5_avg_soilmoisture',
 'pp_day5_cum_dd60',
 'pp_day5_cum_soilmoisture',
    # 'season_length',
    # 'catch_rate',
     'CT_Current',
 # 'WG_Current',
    'year'
]

dftotrain = df_single_batch[cols_to_train]
# dftotrain = dftotrain[(dftotrain.CT_Initial.notna()) & \
# (dftotrain.WG_Initial_7Day_Cnt.notna()) & \
# (dftotrain.irrigation_type.notna()) & \
# (dftotrain.ans_variety.notna()) & \
# (dftotrain.defol_to_harvest.notna())
# ]

dftotrain = dftotrain.dropna()
```

In [26]:
```
dftotrain.dtypes
```

Out [26]:
```
irrigation_type                   object
rm                                 int64
cumulated_dd60                   float64
cumulated_soil_moisture          float64
dd60                             float64
defol_to_harvest                 float64
defol_to_harvest_average_dd60    float64
defol_to_harvest_average_heat    float64
defol_to_harvest_cum_dd60        float64
defol_to_harvest_cum_heat        float64
```

```
maczone                         float64
pp_day10_avg_dd60               float64
pp_day10_avg_soilmoisture       float64
pp_day10_cum_dd60               float64
pp_day10_cum_soilmoisture       float64
pp_day5_avg_dd60                float64
pp_day5_avg_soilmoisture        float64
pp_day5_cum_dd60                float64
pp_day5_cum_soilmoisture        float64
CT_Current                      float64
year                              int32
dtype: object
```

In [ ]:
```
dftotrain.head()
```

In [ ]:
```
dftotrain.isna().sum()
```

In [ ]:
```
# we will use average harvest to ct date across all train
dftotrain.harvest_to_CT_Date.mean()
```

In [88]:
```
# classification
cold_class = {'CT_Current': dftotrain.CT_Current >= 30.0}
```

Begin Add new data from actuals from George

In [ ]:
```
# george complained dated 12/23/2025 that we were predicting too many `below 30s` so it made the model unusabl
# so my fix was to retrain with additional data
```

In [18]:
```
df_actuals = pd.read_excel(r"./feedback/Harvest CT Prediction Model - 12.22.25.xlsx", sheet_name="PA ID Summar
```

In [19]:
```
df_actuals = df_actuals[['Row Labels', 'CT']]
df_actuals = df_actuals[df_actuals.CT.notna()]
df_actuals.head()
```

Out [19]:

	Row Labels	CT
0	1751469	43.666667
1	1684190	50.500000
2	1751476	53.250000
3	1669558	36.250000
4	1751445	59.500000

In [20]:
```
# from the same excel sheet, grab the model features
df_model_for_actual = pd.read_excel(r"./feedback/Harvest CT Prediction Model - 12.22.25.xlsx", sheet_name="Mod
```

In [21]:
```
df_model_for_actual = df_model_for_actual[df_model_for_actual.pa_id.isin(df_actuals['Row Labels'])]
```

In [22]:
```
# grab the row labels to ct as a dict and pass them to df_model
paid_to_ct = df_actuals.set_index('Row Labels')['CT'].to_dict()
df_model_for_actual = df_model_for_actual.copy()
df_model_for_actual['CT_Current'] = df_model_for_actual['pa_id'].map(paid_to_ct)
```

In [23]:
```
df_model_for_actual = df_model_for_actual[dftotrain.columns]
print(dftotrain.shape)
dftotrain = pd.concat([dftotrain, df_model_for_actual], ignore_index=True)
print(dftotrain.shape)
```

```
(698, 21)
(923, 21)
```

End appending data from George

In [25]:
```
# repeating the first half of the next cell so i can use it to generate train test split
from sklearn.model_selection import train_test_split
train_data = dftotrain.copy()

# Binarize target: 'poor' (<30) as 1 (positive/minority), 'good' (>=30) as 0
train_data['label'] = (train_data['CT_Current'] < 30).astype(int)  # 1=poor, 0=good
train_data = train_data.drop('CT_Current', axis=1)
```

```
# Stratified split
train_split, test_split = train_test_split(
    train_data, test_size=0.2, stratify=train_data['label'], random_state=42
)
```

In [80]:

```
import pandas as pd
import os  # For mkdir
from sklearn.model_selection import train_test_split
from autogluon.tabular import TabularPredictor
from sklearn.metrics import classification_report, confusion_matrix

# Load and preprocess data
train_data = dftotrain.copy()

# Binarize target: 'poor' (<30) as 1 (positive/minority), 'good' (>=30) as 0
train_data['label'] = (train_data['CT_Current'] < 30).astype(int)  # 1=poor, 0=good
train_data = train_data.drop('CT_Current', axis=1)

# Stratified split
train_split, test_split = train_test_split(
    train_data, test_size=0.2, stratify=train_data['label'], random_state=42
)

timestamp = pd.Timestamp.now().strftime("%Y%m%d_%H%M%S")
model_path = fr'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-dormancy\notebooks\cotto
os.makedirs(model_path, exist_ok=True)  # Create if missing

# Initialize predictor
predictor = TabularPredictor(
    label='label',
    problem_type='binary',
    eval_metric='f1_macro',
    path=model_path
).fit(
    train_split,
    time_limit=2800,
    presets='best_quality',
    auto_stack=True,
    dynamic_stacking=True  # Bypass DyStack (no temp Ray cleanup needed)
)

# Predict/eval unchanged
test_preds = predictor.predict(test_split.drop('label', axis=1), as_pandas=True)
test_probs = predictor.predict_proba(test_split.drop('label', axis=1), as_pandas=True)
y_true = test_split['label']
print(classification_report(y_true, test_preds, target_names=['good', 'poor']))
print("Confusion Matrix:\n", confusion_matrix(y_true, test_preds))
print(predictor.leaderboard(test_split, silent=True))
```

```
Warning: path already exists! This predictor may overwrite an existing predictor! path="C:\Users\EIJEL\OneDrive - Bayer\Personal
Data\Erik\2024\pkl_new\cotton-dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_115723"
Verbosity: 2 (Standard Logging)
=================== System Info ===================
AutoGluon Version:  1.4.0
Python Version:     3.11.2
Operating System:   Windows
Platform Machine:   AMD64
Platform Version:   10.0.22631
CPU Count:          8
Memory Avail:       2.72 GB / 15.71 GB (17.3%)
Disk Space Avail:   77.96 GB / 237.23 GB (32.9%)
====================================================
Presets specified: ['best_quality']
Using hyperparameters preset: hyperparameters='zeroshot'
Stack configuration (auto_stack=True): num_stack_levels=1, num_bag_folds=8, num_bag_sets=1
DyStack is enabled (dynamic_stacking=True). AutoGluon will try to determine whether the input data is affected by stacked overfitting
and enable or disable stacking as a consequence.
        This is used to identify the optimal `num_stack_levels` value. Copies of AutoGluon will be fit on subsets of the data. Then
holdout validation data is used to detect stacked overfitting.
        Running DyStack for up to 700s of the 2800s of remaining time (25%).
2025-12-23 11:57:24,574 INFO util.py:154 -- Missing packages: ['ipywidgets']. Run `pip install -U ipywidgets`, then restart the
notebook server for rich notebook output.
        Running DyStack sub-fit in a ray process to avoid memory leakage. Enabling ray logging (enable_ray_logging=True). Specify
`ds_args={'enable_ray_logging': False}` if you experience logging issues.
2025-12-23 11:57:34,316 INFO worker.py:1843 -- Started a local Ray instance. View the dashboard at [1m[32mhttp://127.0.0.1:8265
[39m[22m
                Context path: "C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_115723\ds_sub_fit\sub_fit_ho"
[36m(_dystack pid=26284)[0m Running DyStack sub-fit ...
[36m(_dystack pid=26284)[0m Beginning AutoGluon training ... Time limit = 676s
[36m(_dystack pid=26284)[0m AutoGluon will save models to "C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_115723\ds_sub_fit\sub_fit_ho"
[36m(_dystack pid=26284)[0m Train Data Rows:    656
[36m(_dystack pid=26284)[0m Train Data Columns: 20
[36m(_dystack pid=26284)[0m Label Column:       label
[36m(_dystack pid=26284)[0m Problem Type:       binary
```

```
[36m(_dystack pid=26284)[0m Preprocessing data ...
[36m(_dystack pid=26284)[0m Selected class <--> label mapping:  class 1 = 1, class 0 = 0
[36m(_dystack pid=26284)[0m Using Feature Generators to preprocess the data ...
[36m(_dystack pid=26284)[0m Fitting AutoMLPipelineFeatureGenerator...
[36m(_dystack pid=26284)[0m     Available Memory:                    2159.66 MB
[36m(_dystack pid=26284)[0m     Train Data (Original)  Memory Usage: 0.14 MB (0.0% of available memory)
[36m(_dystack pid=26284)[0m     Inferring data type of each feature based on column values. Set feature_metadata_in to manually specify
special dtypes of the features.
[36m(_dystack pid=26284)[0m     Stage 1 Generators:
[36m(_dystack pid=26284)[0m             Fitting AsTypeFeatureGenerator...
[36m(_dystack pid=26284)[0m     Stage 2 Generators:
[36m(_dystack pid=26284)[0m             Fitting FillNaFeatureGenerator...
[36m(_dystack pid=26284)[0m     Stage 3 Generators:
[36m(_dystack pid=26284)[0m             Fitting IdentityFeatureGenerator...
[36m(_dystack pid=26284)[0m             Fitting CategoryFeatureGenerator...
[36m(_dystack pid=26284)[0m                     Fitting CategoryMemoryMinimizeFeatureGenerator...
[36m(_dystack pid=26284)[0m     Stage 4 Generators:
[36m(_dystack pid=26284)[0m             Fitting DropUniqueFeatureGenerator...
[36m(_dystack pid=26284)[0m     Stage 5 Generators:
[36m(_dystack pid=26284)[0m             Fitting DropDuplicatesFeatureGenerator...
[36m(_dystack pid=26284)[0m     Types of features in original data (raw dtype, special dtypes):
[36m(_dystack pid=26284)[0m             ('float', [])  : 17 | ['cumulated_dd60', 'cumulated_soil_moisture', 'dd60', 'defol_to_harvest',
'defol_to_harvest_average_dd60', ...]
[36m(_dystack pid=26284)[0m             ('int', [])    :  2 | ['rm', 'year']
[36m(_dystack pid=26284)[0m             ('object', []) :  1 | ['irrigation_type']
[36m(_dystack pid=26284)[0m     Types of features in processed data (raw dtype, special dtypes):
[36m(_dystack pid=26284)[0m             ('category', []) :  1 | ['irrigation_type']
[36m(_dystack pid=26284)[0m             ('float', [])    : 17 | ['cumulated_dd60', 'cumulated_soil_moisture', 'dd60',
'defol_to_harvest', 'defol_to_harvest_average_dd60', ...]
[36m(_dystack pid=26284)[0m             ('int', [])      :  2 | ['rm', 'year']
[36m(_dystack pid=26284)[0m     0.1s = Fit runtime
[36m(_dystack pid=26284)[0m     20 features in original data used to generate 20 features in processed data.
[36m(_dystack pid=26284)[0m     Train Data (Processed) Memory Usage: 0.10 MB (0.0% of available memory)
[36m(_dystack pid=26284)[0m Data preprocessing and feature engineering runtime = 0.16s ...
[36m(_dystack pid=26284)[0m AutoGluon will gauge predictive performance using evaluation metric: 'f1_macro'
[36m(_dystack pid=26284)[0m     To change this, specify the eval_metric parameter of Predictor()
[36m(_dystack pid=26284)[0m Large model count detected (110 configs) ... Only displaying the first 3 models of each family. To see all,
set `verbosity=3`.
[36m(_dystack pid=26284)[0m User-specified model hyperparameters to be fit:
[36m(_dystack pid=26284)[0m {
[36m(_dystack pid=26284)[0m     'NN_TORCH': [{}, {'activation': 'elu', 'dropout_prob': 0.10077639529843717, 'hidden_size': 108,
'learning_rate': 0.002735937344002146, 'num_layers': 4, 'use_batchnorm': True, 'weight_decay': 1.356433327634438e-12, 'ag_args':
{'name_suffix': '_r79', 'priority': -2}}, {'activation': 'elu', 'dropout_prob': 0.11897478034205347, 'hidden_size': 213,
'learning_rate': 0.0010474382260641949, 'num_layers': 4, 'use_batchnorm': False, 'weight_decay': 5.594471067786272e-10, 'ag_args':
{'name_suffix': '_r22', 'priority': -7}}],
[36m(_dystack pid=26284)[0m     'GBM': [{'extra_trees': True, 'ag_args': {'name_suffix': 'XT'}}, {}, {'learning_rate': 0.03,
'num_leaves': 128, 'feature_fraction': 0.9, 'min_data_in_leaf': 3, 'ag_args': {'name_suffix': 'Large', 'priority': 0,
'hyperparameter_tune_kwargs': None}}],
[36m(_dystack pid=26284)[0m     'CAT': [{}, {'depth': 6, 'grow_policy': 'SymmetricTree', 'l2_leaf_reg': 2.1542798306067823,
'learning_rate': 0.06864209415792857, 'max_ctr_complexity': 4, 'one_hot_max_size': 10, 'ag_args': {'name_suffix': '_r177', 'priority':
-1}}, {'depth': 8, 'grow_policy': 'Depthwise', 'l2_leaf_reg': 2.7997999596449104, 'learning_rate': 0.031375015734637225,
'max_ctr_complexity': 2, 'one_hot_max_size': 3, 'ag_args': {'name_suffix': '_r9', 'priority': -5}}],
[36m(_dystack pid=26284)[0m     'XGB': [{}, {'colsample_bytree': 0.6917311125174739, 'enable_categorical': False, 'learning_rate':
0.018063876087523967, 'max_depth': 10, 'min_child_weight': 0.6028633586934382, 'ag_args': {'name_suffix': '_r33', 'priority': -8}},
{'colsample_bytree': 0.6628423832084077, 'enable_categorical': False, 'learning_rate': 0.08775715546881824, 'max_depth': 5,
'min_child_weight': 0.6294123374222513, 'ag_args': {'name_suffix': '_r89', 'priority': -16}}],
[36m(_dystack pid=26284)[0m     'FASTAI': [{}, {'bs': 256, 'emb_drop': 0.5411770367537934, 'epochs': 43, 'layers': [800, 400], 'lr':
0.01519848858318159, 'ps': 0.2378294656660438 5, 'ag_args': {'name_suffix': '_r191', 'priority': -4}}, {'bs': 2048, 'emb_drop':
0.05070411322605811, 'epochs': 29, 'layers': [200, 100], 'lr': 0.08974235041576624, 'ps': 0.10393466140748028, 'ag_args':
{'name_suffix': '_r102', 'priority': -11}}],
[36m(_dystack pid=26284)[0m     'RF': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary',
'multiclass']}}, {'criterion': 'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion':
'squared_error', 'ag_args': {'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
[36m(_dystack pid=26284)[0m     'XT': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary',
'multiclass']}}, {'criterion': 'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion':
'squared_error', 'ag_args': {'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
[36m(_dystack pid=26284)[0m }
[36m(_dystack pid=26284)[0m AutoGluon will fit 2 stack levels (L1 to L2) ...
[36m(_dystack pid=26284)[0m Fitting 108 L1 models, fit_strategy="sequential" ...
[36m(_dystack pid=26284)[0m Fitting model: LightGBMXT_BAG_L1 ... Training model for up to 450.28s of the 675.57s of remaining time.
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=0.34%)
[36m(_dystack pid=26284)[0m     0.8598   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     13.47s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.19s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: LightGBM_BAG_L1 ... Training model for up to 405.61s of the 630.91s of remaining time.
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=0.35%)
[36m(_dystack pid=26284)[0m     0.8552   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     12.21s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.11s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: RandomForestGini_BAG_L1 ... Training model for up to 371.14s of the 596.43s of remaining
time.
[36m(_dystack pid=26284)[0m     0.8262   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     2.32s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.26s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: RandomForestEntr_BAG_L1 ... Training model for up to 368.43s of the 593.72s of remaining
time.
[36m(_dystack pid=26284)[0m     0.8277   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     1.42s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.29s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: CatBoost_BAG_L1 ... Training model for up to 366.58s of the 591.87s of remaining time.
[36m(_dystack pid=26284)[0m     Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 13.54%
memory usage per fold, 54.17%/80.00% total).
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per:
cpus=2, gpus=0, memory=13.54%)
[36m(_dystack pid=26284)[0m     0.8369   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     133.5s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.05s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: ExtraTreesGini_BAG_L1 ... Training model for up to 217.46s of the 442.75s of remaining time.
[36m(_dystack pid=26284)[0m     0.8399   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     1.21s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.23s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: ExtraTreesEntr_BAG_L1 ... Training model for up to 215.90s of the 441.19s of remaining time.
[36m(_dystack pid=26284)[0m     0.8399   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     1.23s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.23s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: NeuralNetFastAI_BAG_L1 ... Training model for up to 214.34s of the 439.63s of remaining
time.
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=0.04%)
[36m(_ray_fit pid=24232)[0m No improvement since epoch 3: early stopping
[36m(_dystack pid=26284)[0m     0.8521   = Validation score   (f1_macro)
```

```
[36m(_dystack pid=26284)[0m     36.46s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.33s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: XGBoost_BAG_L1 ... Training model for up to 157.18s of the 382.47s of remaining time.
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=1.53%)
[36m(_dystack pid=26284)[0m     0.8476   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     10.41s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.11s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: NeuralNetTorch_BAG_L1 ... Training model for up to 123.60s of the 348.89s of remaining time.
[36m(_ray_fit pid=25996)[0m No improvement since epoch 8: early stopping
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=0.02%)
[36m(_ray_fit pid=24684)[0m C:\Users\EIJEL\AppData\Local\Programs\Python\Python311\Lib\site-
packages\sklearn\compose\_column_transformer.py:975: FutureWarning: The parameter `force_int_remainder_cols` is deprecated and will be
removed in 1.9. It has no effect. Leave it to its default value to avoid this warning.
[36m(_ray_fit pid=24684)[0m   warnings.warn(
[36m(_dystack pid=26284)[0m     0.8415   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     35.02s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.25s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: LightGBMLarge_BAG_L1 ... Training model for up to 71.28s of the 296.57s of remaining time.
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=1.18%)
[36m(_ray_fit pid=29224)[0m C:\Users\EIJEL\AppData\Local\Programs\Python\Python311\Lib\site-
packages\sklearn\compose\_column_transformer.py:975: FutureWarning: The parameter `force_int_remainder_cols` is deprecated and will be
removed in 1.9. It has no effect. Leave it to its default value to avoid this warning.[32m [repeated 7x across cluster] (Ray
deduplicates logs by default. Set RAY_DEDUP_LOGS=0 to disable log deduplication, or see https://docs.ray.io/en/master/ray-
observability/user-guides/configure-logging.html#log-deduplication for more options.)[0m
[36m(_ray_fit pid=29224)[0m   warnings.warn([32m [repeated 7x across cluster][0m
[36m(_dystack pid=26284)[0m     0.8384   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     12.94s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.08s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: CatBoost_r177_BAG_L1 ... Training model for up to 40.48s of the 265.77s of remaining time.
[36m(_dystack pid=26284)[0m     Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 13.77%
memory usage per fold, 55.07%/80.00% total).
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per:
cpus=2, gpus=0, memory=13.77%)
[36m(_ray_fit pid=20796)[0m     Ran out of time, early stopping on iteration 132.
[36m(_ray_fit pid=26684)[0m     Ran out of time, early stopping on iteration 155.[32m [repeated 4x across cluster][0m
[36m(_dystack pid=26284)[0m     0.8247   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     44.86s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.07s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: WeightedEnsemble_L2 ... Training model for up to 360.00s of the 208.19s of remaining time.
[36m(_dystack pid=26284)[0m     Ensemble Weights: {'LightGBMXT_BAG_L1': 1.0}
[36m(_dystack pid=26284)[0m     0.8598   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     0.7s     = Training   runtime
[36m(_dystack pid=26284)[0m     0.0s     = Validation runtime
[36m(_dystack pid=26284)[0m Fitting 108 L2 models, fit_strategy="sequential" ...
[36m(_dystack pid=26284)[0m Fitting model: LightGBMXT_BAG_L2 ... Training model for up to 207.43s of the 207.32s of remaining time.
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=0.29%)
[36m(_dystack pid=26284)[0m     0.8552   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     8.66s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.07s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: LightGBM_BAG_L2 ... Training model for up to 178.66s of the 178.54s of remaining time.
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per:
cpus=1, gpus=0, memory=0.41%)
[36m(_ray_fit pid=18652)[0m     Ran out of time, early stopping on iteration 145.[32m [repeated 3x across cluster][0m
[36m(_dystack pid=26284)[0m     0.8612   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     17.77s   = Training   runtime
[36m(_dystack pid=26284)[0m     0.15s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: RandomForestGini_BAG_L2 ... Training model for up to 139.75s of the 139.63s of remaining
time.
[36m(_dystack pid=26284)[0m     0.843    = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     2.35s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.41s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: RandomForestEntr_BAG_L2 ... Training model for up to 136.56s of the 136.45s of remaining
time.
[36m(_dystack pid=26284)[0m     0.8552   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     2.14s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.37s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: CatBoost_BAG_L2 ... Training model for up to 133.87s of the 133.76s of remaining time.
[36m(_dystack pid=26284)[0m     Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 14.35%
memory usage per fold, 57.40%/80.00% total).
[36m(_dystack pid=26284)[0m     Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per:
cpus=2, gpus=0, memory=14.35%)
[36m(_ray_fit pid=22324)[0m     Ran out of time, early stopping on iteration 395.
[36m(_ray_fit pid=4908)[0m      Ran out of time, early stopping on iteration 377.
[36m(_ray_fit pid=22128)[0m     Ran out of time, early stopping on iteration 394.
[36m(_dystack pid=26284)[0m     0.843    = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     116.05s  = Training   runtime
[36m(_dystack pid=26284)[0m     0.06s    = Validation runtime
[36m(_dystack pid=26284)[0m Fitting model: WeightedEnsemble_L3 ... Training model for up to 360.00s of the 3.15s of remaining time.
[36m(_dystack pid=26284)[0m     Ensemble Weights: {'LightGBM_BAG_L2': 1.0}
[36m(_dystack pid=26284)[0m     0.8612   = Validation score   (f1_macro)
[36m(_dystack pid=26284)[0m     0.63s    = Training   runtime
[36m(_dystack pid=26284)[0m     0.0s     = Validation runtime
[36m(_dystack pid=26284)[0m AutoGluon training complete, total runtime = 673.31s ... Best model: WeightedEnsemble_L3 | Estimated
inference throughput: 71.4 rows/s (82 batch size)
[36m(_dystack pid=26284)[0m Enabling decision threshold calibration (calibrate_decision_threshold='auto', metric is valid, problem_type
is 'binary')
[36m(_dystack pid=26284)[0m Calibrating decision threshold to optimize metric f1_macro | Checking 51 thresholds...
[36m(_dystack pid=26284)[0m Calibrating decision threshold via fine-grained search | Checking 38 thresholds...
[36m(_dystack pid=26284)[0m     Base Threshold: 0.500   | val: 0.8612
[36m(_dystack pid=26284)[0m     Best Threshold: 0.500   | val: 0.8612
[36m(_dystack pid=26284)[0m TabularPredictor saved. To load, use: predictor = TabularPredictor.load("C:\Users\EIJEL\OneDrive -
Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_115723\ds_sub_fit\sub_fit_ho")
[36m(_dystack pid=26284)[0m Deleting DyStack predictor artifacts (clean_up_fits=True) ...
Leaderboard on holdout data (DyStack):
                     model  score_holdout  score_val eval_metric  pred_time_test  pred_time_val    fit_time  pred_time_test_marginal
pred_time_val_marginal  fit_time_marginal  stack_level  can_infer  fit_order
0      CatBoost_r177_BAG_L1       0.841440   0.824691    f1_macro        0.050569       0.069752   44.861721                 0.050569
0.069752          44.861721            1       True         12
1     ExtraTreesEntr_BAG_L1       0.829167   0.839936    f1_macro        0.101232       0.234470    1.227208                 0.101232
0.234470           1.227208            1       True          7
2     ExtraTreesGini_BAG_L1       0.829167   0.839894    f1_macro        0.123467       0.232873    1.210710                 0.123467
0.232873           1.210710            1       True          6
3   RandomForestEntr_BAG_L1       0.829167   0.827724    f1_macro        0.136883       0.293388    1.423090                 0.136883
0.293388           1.423090            1       True          4
4   RandomForestGini_BAG_L1       0.829167   0.826205    f1_macro        0.303913       0.256891    2.323684                 0.303913
0.256891           2.323684            1       True          3
5         LightGBMXT_BAG_L1       0.829167   0.859751    f1_macro        0.711785       0.192446   13.468168                 0.711785
0.192446          13.468168            1       True          1
```

```
6       WeightedEnsemble_L2       0.829167   0.859751    f1_macro        0.715697       0.197359   14.168006                  0.003912
0.004912           0.699837            2        True          13
7           CatBoost_BAG_L1       0.829167   0.836881    f1_macro        1.279509       0.054046  133.502081                  1.279509
0.054046         133.502081            1        True           5
8           CatBoost_BAG_L2       0.829167   0.842979    f1_macro        5.179415       1.524728  347.565611                  0.066235
0.064115         116.050073            2        True          18
9   RandomForestEntr_BAG_L2       0.817046   0.855180    f1_macro        5.235425       1.835362  233.655898                  0.122245
0.374749           2.140359            2        True          17
10        LightGBMXT_BAG_L2       0.817046   0.855183    f1_macro        5.237350       1.534033  240.175541                  0.124171
0.073420           8.660003            2        True          14
11  RandomForestGini_BAG_L2       0.817046   0.842979    f1_macro        5.278836       1.869477  233.869045                  0.165657
0.408864           2.353506            2        True          16
12          LightGBM_BAG_L1       0.816828   0.855183    f1_macro        0.115733       0.105810   12.210052                  0.115733
0.105810          12.210052            1        True           2
13     LightGBMLarge_BAG_L1       0.816828   0.838409    f1_macro        0.314815       0.081397   12.938136                  0.314815
0.081397          12.938136            1        True          11
14           XGBoost_BAG_L1       0.816828   0.847560    f1_macro        0.323761       0.109059   10.413452                  0.323761
0.109059          10.413452            1        True           9
15    NeuralNetTorch_BAG_L1       0.804878   0.841463    f1_macro        0.215093       0.249216   35.024494                  0.215093
0.249216          35.024494            1        True          10
16   NeuralNetFastAI_BAG_L1       0.792652   0.852076    f1_macro        2.344918       0.327987   36.457045                  2.344918
0.327987          36.457045            1        True           8
17          LightGBM_BAG_L2       0.792652   0.861226    f1_macro        5.243255       1.611109  249.287751                  0.130075
0.150496          17.772213            2        True          15
18      WeightedEnsemble_L3       0.792652   0.861226    f1_macro        5.246822       1.611109  249.921883                  0.003567
0.000000           0.634132            3        True          19
        0        = Optimal   num_stack_levels (Stacked Overfitting Occurred: True)
        719s     = DyStack   runtime |  2081s    = Remaining runtime
Starting main fit with num_stack_levels=0.
        For future fit calls on this dataset, you can skip DyStack to save time: `predictor.fit(..., dynamic_stacking=False,
num_stack_levels=0)`
Beginning AutoGluon training ... Time limit = 2081s
AutoGluon will save models to "C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_115723"
Train Data Rows:    738
Train Data Columns: 20
Label Column:       label
Problem Type:       binary
Preprocessing data ...
Selected class <--> label mapping:  class 1 = 1, class 0 = 0
Using Feature Generators to preprocess the data ...
Fitting AutoMLPipelineFeatureGenerator...
        Available Memory:                    3072.94 MB
        Train Data (Original)  Memory Usage: 0.16 MB (0.0% of available memory)
        Inferring data type of each feature based on column values. Set feature_metadata_in to manually specify special dtypes of the
features.
        Stage 1 Generators:
                Fitting AsTypeFeatureGenerator...
        Stage 2 Generators:
                Fitting FillNaFeatureGenerator...
        Stage 3 Generators:
                Fitting IdentityFeatureGenerator...
                Fitting CategoryFeatureGenerator...
                        Fitting CategoryMemoryMinimizeFeatureGenerator...
        Stage 4 Generators:
                Fitting DropUniqueFeatureGenerator...
        Stage 5 Generators:
                Fitting DropDuplicatesFeatureGenerator...
        Types of features in original data (raw dtype, special dtypes):
                ('float', [])  : 17 | ['cumulated_dd60', 'cumulated_soil_moisture', 'dd60', 'defol_to_harvest',
'defol_to_harvest_average_dd60', ...]
                ('int', [])    :  2 | ['rm', 'year']
                ('object', []) :  1 | ['irrigation_type']
        Types of features in processed data (raw dtype, special dtypes):
                ('category', []) :  1 | ['irrigation_type']
                ('float', [])    : 17 | ['cumulated_dd60', 'cumulated_soil_moisture', 'dd60', 'defol_to_harvest',
'defol_to_harvest_average_dd60', ...]
                ('int', [])      :  2 | ['rm', 'year']
        0.2s = Fit runtime
        20 features in original data used to generate 20 features in processed data.
        Train Data (Processed) Memory Usage: 0.11 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.27s ...
AutoGluon will gauge predictive performance using evaluation metric: 'f1_macro'
        To change this, specify the eval_metric parameter of Predictor()
Large model count detected (110 configs) ... Only displaying the first 3 models of each family. To see all, set `verbosity=3`.
User-specified model hyperparameters to be fit:
{
        'NN_TORCH': [{}, {'activation': 'elu', 'dropout_prob': 0.10077639529843717, 'hidden_size': 108, 'learning_rate':
0.002735937344002146, 'num_layers': 4, 'use_batchnorm': True, 'weight_decay': 1.356433327634438e-12, 'ag_args': {'name_suffix': '_r79',
'priority': -2}}, {'activation': 'elu', 'dropout_prob': 0.11897478034205347, 'hidden_size': 213, 'learning_rate':
0.0010474382260641949, 'num_layers': 4, 'use_batchnorm': False, 'weight_decay': 5.594471067786272e-10, 'ag_args': {'name_suffix':
'_r22', 'priority': -7}}],
        'GBM': [{'extra_trees': True, 'ag_args': {'name_suffix': 'XT'}}, {}, {'learning_rate': 0.03, 'num_leaves': 128,
'feature_fraction': 0.9, 'min_data_in_leaf': 3, 'ag_args': {'name_suffix': 'Large', 'priority': 0, 'hyperparameter_tune_kwargs':
None}}],
        'CAT': [{}, {'depth': 6, 'grow_policy': 'SymmetricTree', 'l2_leaf_reg': 2.1542798306067823, 'learning_rate':
0.06864209415792857, 'max_ctr_complexity': 4, 'one_hot_max_size': 10, 'ag_args': {'name_suffix': '_r177', 'priority': -1}}, {'depth':
8, 'grow_policy': 'Depthwise', 'l2_leaf_reg': 2.7997999596449104, 'learning_rate': 0.031375015734637225, 'max_ctr_complexity': 2,
'one_hot_max_size': 3, 'ag_args': {'name_suffix': '_r9', 'priority': -5}}],
        'XGB': [{}, {'colsample_bytree': 0.6917311125174739, 'enable_categorical': False, 'learning_rate': 0.018063876087523967,
'max_depth': 10, 'min_child_weight': 0.6028633586934382, 'ag_args': {'name_suffix': '_r33', 'priority': -8}}, {'colsample_bytree':
0.6628423832084077, 'enable_categorical': False, 'learning_rate': 0.08775715546881824, 'max_depth': 5, 'min_child_weight':
0.6294123374222513, 'ag_args': {'name_suffix': '_r89', 'priority': -16}}],
        'FASTAI': [{}, {'bs': 256, 'emb_drop': 0.5411770367537934, 'epochs': 43, 'layers': [800, 400], 'lr': 0.01519848858318159, 'ps':
0.23782946566604385, 'ag_args': {'name_suffix': '_r191', 'priority': -4}}, {'bs': 2048, 'emb_drop': 0.05070411322605811, 'epochs': 29,
'layers': [200, 100], 'lr': 0.08974235041576624, 'ps': 0.10393466140748028, 'ag_args': {'name_suffix': '_r102', 'priority': -11}}],
        'RF': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion':
'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args':
{'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
        'XT': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion':
'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args':
{'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
}
Fitting 108 L1 models, fit_strategy="sequential" ...
Fitting model: LightGBMXT_BAG_L1 ... Training model for up to 2081.17s of the 2081.13s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.22%)
        0.8564   = Validation score   (f1_macro)
        13.16s   = Training   runtime
        0.12s    = Validation runtime
Fitting model: LightGBM_BAG_L1 ... Training model for up to 2047.52s of the 2047.48s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.24%)
```

```
        0.8414   = Validation score   (f1_macro)
        8.58s    = Training   runtime
        0.07s    = Validation runtime
Fitting model: RandomForestGini_BAG_L1 ... Training model for up to 2014.60s of the 2014.56s of remaining time.
        0.8265   = Validation score   (f1_macro)
        1.69s    = Training   runtime
        0.29s    = Validation runtime
Fitting model: RandomForestEntr_BAG_L1 ... Training model for up to 2012.50s of the 2012.46s of remaining time.
        0.8292   = Validation score   (f1_macro)
        1.49s    = Training   runtime
        0.26s    = Validation runtime
Fitting model: CatBoost_BAG_L1 ... Training model for up to 2010.64s of the 2010.60s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 12.18% memory usage per fold,
48.70%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=12.18%)
        0.8306   = Validation score   (f1_macro)
        100.76s  = Training   runtime
        0.05s    = Validation runtime
Fitting model: ExtraTreesGini_BAG_L1 ... Training model for up to 1898.16s of the 1898.12s of remaining time.
        0.8387   = Validation score   (f1_macro)
        0.96s    = Training   runtime
        0.1s     = Validation runtime
Fitting model: ExtraTreesEntr_BAG_L1 ... Training model for up to 1897.03s of the 1896.99s of remaining time.
        0.8347   = Validation score   (f1_macro)
        1.27s    = Training   runtime
        0.13s    = Validation runtime
Fitting model: NeuralNetFastAI_BAG_L1 ... Training model for up to 1895.54s of the 1895.50s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.04%)
        0.8482   = Validation score   (f1_macro)
        22.31s   = Training   runtime
        0.24s    = Validation runtime
Fitting model: XGBoost_BAG_L1 ... Training model for up to 1857.79s of the 1857.74s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=1.13%)
        0.8333   = Validation score   (f1_macro)
        8.37s    = Training   runtime
        0.12s    = Validation runtime
Fitting model: NeuralNetTorch_BAG_L1 ... Training model for up to 1831.13s of the 1831.09s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.02%)
        0.8428   = Validation score   (f1_macro)
        40.28s   = Training   runtime
        0.18s    = Validation runtime
Fitting model: LightGBMLarge_BAG_L1 ... Training model for up to 1774.79s of the 1774.75s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.80%)
        0.8333   = Validation score   (f1_macro)
        13.84s   = Training   runtime
        0.1s     = Validation runtime
Fitting model: CatBoost_r177_BAG_L1 ... Training model for up to 1745.03s of the 1744.99s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 11.53% memory usage per fold,
46.13%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=11.53%)
        0.8266   = Validation score   (f1_macro)
        92.41s   = Training   runtime
        0.05s    = Validation runtime
Fitting model: NeuralNetTorch_r79_BAG_L1 ... Training model for up to 1639.30s of the 1639.26s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.02%)
        0.855    = Validation score   (f1_macro)
        45.27s   = Training   runtime
        0.27s    = Validation runtime
Fitting model: LightGBM_r131_BAG_L1 ... Training model for up to 1578.06s of the 1578.01s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.31%)
        0.8509   = Validation score   (f1_macro)
        10.46s   = Training   runtime
        0.03s    = Validation runtime
Fitting model: NeuralNetFastAI_r191_BAG_L1 ... Training model for up to 1550.38s of the 1550.34s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.03%)
        0.8482   = Validation score   (f1_macro)
        29.92s   = Training   runtime
        0.35s    = Validation runtime
Fitting model: CatBoost_r9_BAG_L1 ... Training model for up to 1503.56s of the 1503.52s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 2 folds in parallel instead (Estimated 25.02% memory usage per fold,
50.03%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (2 workers, per: cpus=4, gpus=0,
memory=25.02%)
        0.8388   = Validation score   (f1_macro)
        222.05s  = Training   runtime
        0.07s    = Validation runtime
Fitting model: LightGBM_r96_BAG_L1 ... Training model for up to 1271.11s of the 1271.07s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.11%)
        0.8347   = Validation score   (f1_macro)
        7.49s    = Training   runtime
        0.06s    = Validation runtime
Fitting model: NeuralNetTorch_r22_BAG_L1 ... Training model for up to 1246.86s of the 1246.82s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.02%)
        0.8522   = Validation score   (f1_macro)
        41.17s   = Training   runtime
        0.34s    = Validation runtime
Fitting model: XGBoost_r33_BAG_L1 ... Training model for up to 1189.42s of the 1189.38s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=1.97%)
        0.8414   = Validation score   (f1_macro)
        14.45s   = Training   runtime
        0.17s    = Validation runtime
Fitting model: ExtraTrees_r42_BAG_L1 ... Training model for up to 1152.16s of the 1152.12s of remaining time.
        0.8332   = Validation score   (f1_macro)
        1.64s    = Training   runtime
        0.27s    = Validation runtime
Fitting model: CatBoost_r137_BAG_L1 ... Training model for up to 1150.10s of the 1150.06s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=9.96%)
        0.8414   = Validation score   (f1_macro)
        59.24s   = Training   runtime
        0.06s    = Validation runtime
```

```
Fitting model: NeuralNetFastAI_r102_BAG_L1 ... Training model for up to 1070.72s of the 1070.68s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.03%)
        0.8564   = Validation score   (f1_macro)
        43.32s   = Training   runtime
        0.38s    = Validation runtime
Fitting model: CatBoost_r13_BAG_L1 ... Training model for up to 1008.19s of the 1008.14s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 2 folds in parallel instead (Estimated 21.65% memory usage per fold,
43.30%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (2 workers, per: cpus=4, gpus=0,
memory=21.65%)
        0.8415   = Validation score   (f1_macro)
        301.98s  = Training   runtime
        0.06s    = Validation runtime
Fitting model: RandomForest_r195_BAG_L1 ... Training model for up to 695.31s of the 695.27s of remaining time.
        0.8116   = Validation score   (f1_macro)
        1.48s    = Training   runtime
        0.23s    = Validation runtime
Fitting model: LightGBM_r188_BAG_L1 ... Training model for up to 693.45s of the 693.41s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.69%)
        0.8537   = Validation score   (f1_macro)
        10.13s   = Training   runtime
        0.12s    = Validation runtime
Fitting model: NeuralNetFastAI_r145_BAG_L1 ... Training model for up to 659.91s of the 659.87s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.04%)
        0.8563   = Validation score   (f1_macro)
        50.58s   = Training   runtime
        0.32s    = Validation runtime
Fitting model: XGBoost_r89_BAG_L1 ... Training model for up to 582.17s of the 582.13s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.65%)
        0.8455   = Validation score   (f1_macro)
        13.95s   = Training   runtime
        0.1s     = Validation runtime
Fitting model: NeuralNetTorch_r30_BAG_L1 ... Training model for up to 547.39s of the 547.35s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.02%)
        0.8576   = Validation score   (f1_macro)
        67.63s   = Training   runtime
        0.34s    = Validation runtime
Fitting model: LightGBM_r130_BAG_L1 ... Training model for up to 456.47s of the 456.43s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.74%)
        0.8591   = Validation score   (f1_macro)
        10.54s   = Training   runtime
        0.09s    = Validation runtime
Fitting model: NeuralNetTorch_r86_BAG_L1 ... Training model for up to 421.09s of the 421.05s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.02%)
        0.8401   = Validation score   (f1_macro)
        52.15s   = Training   runtime
        0.46s    = Validation runtime
Fitting model: CatBoost_r50_BAG_L1 ... Training model for up to 349.99s of the 349.95s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 13.01% memory usage per fold,
52.04%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=13.01%)
        0.8266   = Validation score   (f1_macro)
        110.24s  = Training   runtime
        0.09s    = Validation runtime
Fitting model: NeuralNetFastAI_r11_BAG_L1 ... Training model for up to 219.64s of the 219.60s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.04%)
        0.8523   = Validation score   (f1_macro)
        41.73s   = Training   runtime
        0.63s    = Validation runtime
Fitting model: XGBoost_r194_BAG_L1 ... Training model for up to 154.88s of the 154.83s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=2.32%)
        0.8455   = Validation score   (f1_macro)
        13.31s   = Training   runtime
        0.1s     = Validation runtime
Fitting model: ExtraTrees_r172_BAG_L1 ... Training model for up to 116.43s of the 116.39s of remaining time.
        0.8388   = Validation score   (f1_macro)
        2.07s    = Training   runtime
        0.3s     = Validation runtime
Fitting model: CatBoost_r69_BAG_L1 ... Training model for up to 113.94s of the 113.90s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 11.56% memory usage per fold,
46.23%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=11.56%)
        0.8469   = Validation score   (f1_macro)
        102.45s  = Training   runtime
        0.07s    = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 360.00s of the -3.01s of remaining time.
        Ensemble Weights: {'LightGBM_r130_BAG_L1': 1.0}
        0.8591   = Validation score   (f1_macro)
        0.86s    = Training   runtime
        0.0s     = Validation runtime
AutoGluon training complete, total runtime = 2085.43s ... Best model: WeightedEnsemble_L2 | Estimated inference throughput: 1014.3
rows/s (93 batch size)
Enabling decision threshold calibration (calibrate_decision_threshold='auto', metric is valid, problem_type is 'binary')
Calibrating decision threshold to optimize metric f1_macro | Checking 51 thresholds...
Calibrating decision threshold via fine-grained search | Checking 38 thresholds...
        Base Threshold: 0.500   | val: 0.8591
        Best Threshold: 0.500   | val: 0.8591
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("C:\Users\EIJEL\OneDrive - Bayer\Personal
Data\Erik\2024\pkl_new\cotton-dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_115723")
```

```
              precision    recall  f1-score   support

        good       0.82      0.82      0.82        95
        poor       0.81      0.81      0.81        90

    accuracy                           0.82       185
   macro avg       0.82      0.82      0.82       185
weighted avg       0.82      0.82      0.82       185

Confusion Matrix:
 [[78 17]
```

```
 [17 73]]
                          model  score_test  score_val eval_metric  \
0         ExtraTreesGini_BAG_L1    0.821538   0.838729    f1_macro
1     NeuralNetTorch_r86_BAG_L1    0.816168   0.840079    f1_macro
2         ExtraTreesEntr_BAG_L1    0.816082   0.834658    f1_macro
3          LightGBM_r130_BAG_L1    0.816082   0.859069    f1_macro
4           WeightedEnsemble_L2    0.816082   0.859069    f1_macro
5        ExtraTrees_r172_BAG_L1    0.810789   0.838751    f1_macro
6         ExtraTrees_r42_BAG_L1    0.810722   0.833245    f1_macro
7         NeuralNetTorch_BAG_L1    0.810722   0.842808    f1_macro
8           CatBoost_r13_BAG_L1    0.800000   0.841463    f1_macro
9             LightGBMXT_BAG_L1    0.799977   0.856368    f1_macro
10    NeuralNetTorch_r79_BAG_L1    0.799789   0.855013    f1_macro
11  NeuralNetFastAI_r102_BAG_L1    0.794589   0.856368    f1_macro
12   NeuralNetFastAI_r11_BAG_L1    0.789189   0.852303    f1_macro
13         CatBoost_r177_BAG_L1    0.789165   0.826553    f1_macro
14         LightGBM_r188_BAG_L1    0.789165   0.853659    f1_macro
15    NeuralNetTorch_r22_BAG_L1    0.789165   0.852206    f1_macro
16  NeuralNetFastAI_r145_BAG_L1    0.789165   0.856331    f1_macro
17          XGBoost_r194_BAG_L1    0.788967   0.845510    f1_macro
18          CatBoost_r69_BAG_L1    0.783777   0.846881    f1_macro
19         CatBoost_r137_BAG_L1    0.783777   0.841440    f1_macro
20      RandomForestEntr_BAG_L1    0.783777   0.829248    f1_macro
21    NeuralNetTorch_r30_BAG_L1    0.783777   0.857629    f1_macro
22          CatBoost_r50_BAG_L1    0.783727   0.826558    f1_macro
23           CatBoost_r9_BAG_L1    0.783727   0.838753    f1_macro
24  NeuralNetFastAI_r191_BAG_L1    0.783727   0.848228    f1_macro
25      RandomForestGini_BAG_L1    0.783626   0.826512    f1_macro
26               CatBoost_BAG_L1    0.778352   0.830623    f1_macro
27       NeuralNetFastAI_BAG_L1    0.778275   0.848228    f1_macro
28           XGBoost_r89_BAG_L1    0.778275   0.845527    f1_macro
29         LightGBM_r131_BAG_L1    0.778145   0.850909    f1_macro
30              LightGBM_BAG_L1    0.777963   0.841440    f1_macro
31               XGBoost_BAG_L1    0.772807   0.833309    f1_macro
32          LightGBM_r96_BAG_L1    0.767459   0.834688    f1_macro
33           XGBoost_r33_BAG_L1    0.767459   0.841358    f1_macro
34         LightGBMLarge_BAG_L1    0.756501   0.833318    f1_macro
35     RandomForest_r195_BAG_L1    0.740472   0.811625    f1_macro

    pred_time_test  pred_time_val    fit_time  pred_time_test_marginal  \
0         0.137775       0.102417    0.960597                 0.137775
1         0.655204       0.461594   52.146872                 0.655204
2         0.123343       0.128233    1.270622                 0.123343
3         0.296886       0.091689   10.539958                 0.296886
4         0.301904       0.091689   11.397763                 0.005018
5         0.187919       0.301676    2.071762                 0.187919
6         0.161110       0.270761    1.642359                 0.161110
7         0.263173       0.183474   40.277780                 0.263173
8         0.129005       0.060407  301.981133                 0.129005
9         0.202970       0.115184   13.164535                 0.202970
10        0.295330       0.266289   45.268992                 0.295330
11        0.521239       0.376699   43.320307                 0.521239
12        0.634757       0.631971   41.732490                 0.634757
13        0.083104       0.046750   92.409430                 0.083104
14        0.215324       0.120706   10.129707                 0.215324
15        0.316371       0.344163   41.166116                 0.316371
16        0.495974       0.315087   50.584006                 0.495974
17        0.419663       0.096671   13.305743                 0.419663
18        0.067642       0.070711  102.447420                 0.067642
19        0.086986       0.061750   59.240401                 0.086986
20        0.130101       0.259017    1.489159                 0.130101
21        0.552490       0.340496   67.629629                 0.552490
22        0.103078       0.085380  110.243189                 0.103078
23        0.129810       0.069816  222.045546                 0.129810
24        0.426342       0.349257   29.924247                 0.426342
25        0.174128       0.285941    1.688970                 0.174128
26        0.108471       0.048480  100.755139                 0.108471
27        0.376933       0.240101   22.313913                 0.376933
28        0.543645       0.098028   13.952414                 0.543645
29        0.282384       0.034428   10.462522                 0.282384
30        0.109393       0.074558    8.581465                 0.109393
31        0.314972       0.116848    8.369195                 0.314972
32        0.116170       0.056143    7.488033                 0.116170
33        0.562090       0.167566   14.451835                 0.562090
34        0.352254       0.098371   13.842388                 0.352254
35        0.186989       0.231468    1.478459                 0.186989

    pred_time_val_marginal  fit_time_marginal  stack_level  can_infer  \
0                 0.102417           0.960597            1       True
1                 0.461594          52.146872            1       True
2                 0.128233           1.270622            1       True
3                 0.091689          10.539958            1       True
4                 0.000000           0.857805            2       True
5                 0.301676           2.071762            1       True
6                 0.270761           1.642359            1       True
7                 0.183474          40.277780            1       True
8                 0.060407         301.981133            1       True
9                 0.115184          13.164535            1       True
10                0.266289          45.268992            1       True
11                0.376699          43.320307            1       True
12                0.631971          41.732490            1       True
13                0.046750          92.409430            1       True
14                0.120706          10.129707            1       True
15                0.344163          41.166116            1       True
16                0.315087          50.584006            1       True
17                0.096671          13.305743            1       True
18                0.070711         102.447420            1       True
19                0.061750          59.240401            1       True
20                0.259017           1.489159            1       True
21                0.340496          67.629629            1       True
22                0.085380         110.243189            1       True
23                0.069816         222.045546            1       True
24                0.349257          29.924247            1       True
25                0.285941           1.688970            1       True
26                0.048480         100.755139            1       True
27                0.240101          22.313913            1       True
28                0.098028          13.952414            1       True
29                0.034428          10.462522            1       True
30                0.074558           8.581465            1       True
31                0.116848           8.369195            1       True
32                0.056143           7.488033            1       True
33                0.167566          14.451835            1       True
34                0.098371          13.842388            1       True
```

```
35                0.231468           1.478459            1       True

    fit_order
0           6
1          30
2           7
3          29
4          36
5          34
6          20
7          10
8          23
9           1
10         13
11         22
12         32
13         12
14         25
15         18
16         26
17         33
18         35
19         21
20          4
21         28
22         31
23         16
24         15
25          3
26          5
27          8
28         27
29         14
30          2
31          9
32         17
33         19
34         11
35         24
```

In [81]:

```
################# regression
import pandas as pd
import os  # For mkdir
from sklearn.model_selection import train_test_split
from autogluon.tabular import TabularPredictor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load and preprocess data
train_data = dftotrain.copy()

# For regression: Use 'CT_Current' directly as continuous target (no binarizing)
# Drop if any non-feature cols, but keep 'CT_Current' as label
# Optional: Handle outliers/NaNs if needed, e.g., train_data = train_data.dropna(subset=['CT_Current'])

# Random split (no stratification for regression)
train_split, test_split = train_test_split(
    train_data, test_size=0.2, random_state=42  # Remove stratify
)

timestamp = pd.Timestamp.now().strftime("%Y%m%d_%H%M%S")
model_path = model_path = fr'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-dormancy\no
os.makedirs(model_path, exist_ok=True)  # Create if missing

# Initialize predictor for regression
predictor = TabularPredictor(
    label='CT_Current',  # Direct target column
    problem_type='regression',  # Changed to regression
    eval_metric='mean_absolute_error',  # MAE for score prediction; alternatives: 'root_mean_squared_error' (RM
    path=model_path
).fit(
    train_split,  # Full DF with features + label
    time_limit=2800,  # Same limit
    presets='best_quality',  # Thorough for regression (trees/NNs)
    auto_stack=True,  # Ensembling for better predictions
    dynamic_stacking=True  # Auto-detect stacking viability
)
```

```
Warning: path already exists! This predictor may overwrite an existing predictor! path="C:\Users\EIJEL\OneDrive - Bayer\Personal
Data\Erik\2024\pkl_new\cotton-dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_124744"
Verbosity: 2 (Standard Logging)
================== System Info ==================
AutoGluon Version:  1.4.0
Python Version:     3.11.2
Operating System:   Windows
Platform Machine:   AMD64
Platform Version:   10.0.22631
CPU Count:          8
Memory Avail:       3.24 GB / 15.71 GB (20.6%)
Disk Space Avail:   76.31 GB / 237.23 GB (32.2%)
=================================================
Presets specified: ['best_quality']
Using hyperparameters preset: hyperparameters='zeroshot'
```

```
Stack configuration (auto_stack=True): num_stack_levels=1, num_bag_folds=8, num_bag_sets=1
DyStack is enabled (dynamic_stacking=True). AutoGluon will try to determine whether the input data is affected by stacked overfitting
and enable or disable stacking as a consequence.
        This is used to identify the optimal `num_stack_levels` value. Copies of AutoGluon will be fit on subsets of the data. Then
holdout validation data is used to detect stacked overfitting.
        Running DyStack for up to 700s of the 2800s of remaining time (25%).
                Context path: "C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_124744\ds_sub_fit\sub_fit_ho"
Leaderboard on holdout data (DyStack):
                    model  score_holdout  score_val          eval_metric  pred_time_test  pred_time_val    fit_time
pred_time_test_marginal  pred_time_val_marginal  fit_time_marginal  stack_level  can_infer  fit_order
0      WeightedEnsemble_L3     -10.685020 -10.491389  mean_absolute_error        6.611434       1.899083  408.304393
0.004526                0.002041           0.163235           3       True         15
1          CatBoost_BAG_L2     -10.740728 -10.545411  mean_absolute_error        6.417285       1.582205  407.168281
0.099867                0.068574          97.371517           2       True         12
2     ExtraTreesMSE_BAG_L2     -10.757051 -10.687402  mean_absolute_error        6.507041       1.828468  310.769641
0.189623                0.314837           0.972877           2       True         13
3   RandomForestMSE_BAG_L2     -10.946047 -10.826798  mean_absolute_error        6.522778       1.788698  311.626197
0.205360                0.275067           1.829432           2       True         11
4      WeightedEnsemble_L2     -10.957477 -10.618154  mean_absolute_error        2.502044       0.601115  268.174133
0.007518                0.002020           0.163179           2       True          8
5   RandomForestMSE_BAG_L1     -10.986407 -10.902674  mean_absolute_error        0.259623       0.374603    2.409307
0.259623                0.374603           2.409307           1       True          3
6     ExtraTreesMSE_BAG_L1     -11.013015 -10.822800  mean_absolute_error        0.220985       0.394374    1.259939
0.220985                0.394374           1.259939           1       True          5
7        LightGBMXT_BAG_L2     -11.026921 -10.829103  mean_absolute_error        6.899318       1.850931  337.023297
0.581900                0.337300          27.226533           2       True          9
8           XGBoost_BAG_L1     -11.032970 -11.292437  mean_absolute_error        0.508662       0.118918   11.225731
0.508662                0.118918          11.225731           1       True          7
9          CatBoost_BAG_L1     -11.123935 -10.911699  mean_absolute_error        2.091588       0.109897  255.006222
2.091588                0.109897         255.006222           1       True          4
10         LightGBM_BAG_L1     -11.124722 -10.875638  mean_absolute_error        0.181953       0.094824   11.744793
0.181953                0.094824          11.744793           1       True          2
11         LightGBM_BAG_L2     -11.178711 -10.969672  mean_absolute_error        6.532974       1.626366  318.754786
0.215556                0.112735           8.958022           2       True         10
12       LightGBMXT_BAG_L1     -11.375720 -10.998985  mean_absolute_error        1.209355       0.262831   13.188400
1.209355                0.262831          13.188400           1       True          1
13  NeuralNetFastAI_BAG_L2     -11.708677 -11.117661  mean_absolute_error        6.533442       1.847569  341.180387
0.216024                0.333938          31.383623           2       True         14
14  NeuralNetFastAI_BAG_L1     -13.032732 -11.596904  mean_absolute_error        3.054607       0.421014   28.150772
3.054607                0.421014          28.150772           1       True          6
        1        = Optimal   num_stack_levels (Stacked Overfitting Occurred: False)
        766s     = DyStack   runtime |  2034s    = Remaining runtime
Starting main fit with num_stack_levels=1.
        For future fit calls on this dataset, you can skip DyStack to save time: `predictor.fit(..., dynamic_stacking=False,
num_stack_levels=1)`
Beginning AutoGluon training ... Time limit = 2034s
AutoGluon will save models to "C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_124744"
Train Data Rows:    738
Train Data Columns: 20
Label Column:       CT_Current
Problem Type:       regression
Preprocessing data ...
Using Feature Generators to preprocess the data ...
Fitting AutoMLPipelineFeatureGenerator...
        Available Memory:                    2659.34 MB
        Train Data (Original)  Memory Usage: 0.16 MB (0.0% of available memory)
        Inferring data type of each feature based on column values. Set feature_metadata_in to manually specify special dtypes of the
features.
        Stage 1 Generators:
                Fitting AsTypeFeatureGenerator...
        Stage 2 Generators:
                Fitting FillNaFeatureGenerator...
        Stage 3 Generators:
                Fitting IdentityFeatureGenerator...
                Fitting CategoryFeatureGenerator...
                        Fitting CategoryMemoryMinimizeFeatureGenerator...
        Stage 4 Generators:
                Fitting DropUniqueFeatureGenerator...
        Stage 5 Generators:
                Fitting DropDuplicatesFeatureGenerator...
        Types of features in original data (raw dtype, special dtypes):
                ('float', [])  : 17 | ['cumulated_dd60', 'cumulated_soil_moisture', 'dd60', 'defol_to_harvest',
'defol_to_harvest_average_dd60', ...]
                ('int', []')    :  2 | ['rm', 'year']
                ('object', []) :  1 | ['irrigation_type']
        Types of features in processed data (raw dtype, special dtypes):
                ('category', []) :  1 | ['irrigation_type']
                ('float', []')    : 17 | ['cumulated_dd60', 'cumulated_soil_moisture', 'dd60', 'defol_to_harvest',
'defol_to_harvest_average_dd60', ...]
                ('int', [])      :  2 | ['rm', 'year']
        0.1s = Fit runtime
        20 features in original data used to generate 20 features in processed data.
        Train Data (Processed) Memory Usage: 0.11 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.19s ...
AutoGluon will gauge predictive performance using evaluation metric: 'mean_absolute_error'
        This metric's sign has been flipped to adhere to being higher_is_better. The metric score can be multiplied by -1 to get the
metric value.
        To change this, specify the eval_metric parameter of Predictor()
Large model count detected (110 configs) ... Only displaying the first 3 models of each family. To see all, set `verbosity=3`.
User-specified model hyperparameters to be fit:
{
        'NN_TORCH': [{}, {'activation': 'elu', 'dropout_prob': 0.10077639529843717, 'hidden_size': 108, 'learning_rate':
0.002735937344002146, 'num_layers': 4, 'use_batchnorm': True, 'weight_decay': 1.356433327634438e-12, 'ag_args': {'name_suffix': '_r79',
'priority': -2}}, {'activation': 'elu', 'dropout_prob': 0.11897478034205347, 'hidden_size': 213, 'learning_rate':
0.0010474382260641949, 'num_layers': 4, 'use_batchnorm': False, 'weight_decay': 5.594471067786272e-10, 'ag_args': {'name_suffix':
'_r22', 'priority': -7}}],
        'GBM': [{'extra_trees': True, 'ag_args': {'name_suffix': 'XT'}}, {}, {'learning_rate': 0.03, 'num_leaves': 128,
'feature_fraction': 0.9, 'min_data_in_leaf': 3, 'ag_args': {'name_suffix': 'Large', 'priority': 0, 'hyperparameter_tune_kwargs':
None}}],
        'CAT': [{}, {'depth': 6, 'grow_policy': 'SymmetricTree', 'l2_leaf_reg': 2.1542798306067823, 'learning_rate':
0.06864209415792857, 'max_ctr_complexity': 4, 'one_hot_max_size': 10, 'ag_args': {'name_suffix': '_r177', 'priority': -1}}, {'depth':
8, 'grow_policy': 'Depthwise', 'l2_leaf_reg': 2.7997999596449104, 'learning_rate': 0.031375015734637225, 'max_ctr_complexity': 2,
'one_hot_max_size': 3, 'ag_args': {'name_suffix': '_r9', 'priority': -5}}],
        'XGB': [{}, {'colsample_bytree': 0.6917311125174739, 'enable_categorical': False, 'learning_rate': 0.018063876087523967,
'max_depth': 10, 'min_child_weight': 0.6028633586934382, 'ag_args': {'name_suffix': '_r33', 'priority': -8}}, {'colsample_bytree':
0.6628423832084077, 'enable_categorical': False, 'learning_rate': 0.08775715546881824, 'max_depth': 5, 'min_child_weight':
0.6294123374222513, 'ag_args': {'name_suffix': '_r89', 'priority': -16}}],
        'FASTAI': [{}, {'bs': 256, 'emb_drop': 0.5411770367537934, 'epochs': 43, 'layers': [800, 400], 'lr': 0.01519848858318159, 'ps':
0.23782946566604385, 'ag_args': {'name_suffix': '_r191', 'priority': -4}}, {'bs': 2048, 'emb_drop': 0.05070411322605811, 'epochs': 29,
'layers': [200, 100], 'lr': 0.08974235041576624, 'ps': 0.10393466140748028, 'ag_args': {'name_suffix': '_r102', 'priority': -11}}],
        'RF': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion':
```

```
'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args':
{'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
        'XT': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion':
'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args':
{'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
}
AutoGluon will fit 2 stack levels (L1 to L2) ...
Fitting 106 L1 models, fit_strategy="sequential" ...
Fitting model: LightGBMXT_BAG_L1 ... Training model for up to 1355.55s of the 2033.81s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.27%)
        -10.4833         = Validation score   (-mean_absolute_error)
        18.87s   = Training   runtime
        0.31s    = Validation runtime
Fitting model: LightGBM_BAG_L1 ... Training model for up to 1312.99s of the 1991.25s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.30%)
        -10.7139         = Validation score   (-mean_absolute_error)
        6.63s    = Training   runtime
        0.1s     = Validation runtime
Fitting model: RandomForestMSE_BAG_L1 ... Training model for up to 1288.54s of the 1966.80s of remaining time.
        -11.0369         = Validation score   (-mean_absolute_error)
        1.89s    = Training   runtime
        0.31s    = Validation runtime
Fitting model: CatBoost_BAG_L1 ... Training model for up to 1286.21s of the 1964.47s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 15.42% memory usage per fold,
61.67%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=15.42%)
        -10.7807         = Validation score   (-mean_absolute_error)
        330.37s  = Training   runtime
        0.06s    = Validation runtime
Fitting model: ExtraTreesMSE_BAG_L1 ... Training model for up to 943.27s of the 1621.53s of remaining time.
        -10.73   = Validation score   (-mean_absolute_error)
        0.69s    = Training   runtime
        0.19s    = Validation runtime
Fitting model: NeuralNetFastAI_BAG_L1 ... Training model for up to 942.27s of the 1620.53s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.05%)
        -11.6058         = Validation score   (-mean_absolute_error)
        30.09s   = Training   runtime
        0.36s    = Validation runtime
Fitting model: XGBoost_BAG_L1 ... Training model for up to 893.01s of the 1571.27s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=1.53%)
        -11.0224         = Validation score   (-mean_absolute_error)
        12.48s   = Training   runtime
        0.15s    = Validation runtime
Fitting model: NeuralNetTorch_BAG_L1 ... Training model for up to 856.07s of the 1534.33s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.02%)
        -10.9644         = Validation score   (-mean_absolute_error)
        94.98s   = Training   runtime
        0.28s    = Validation runtime
Fitting model: LightGBMLarge_BAG_L1 ... Training model for up to 742.41s of the 1420.67s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=1.16%)
        -11.1837         = Validation score   (-mean_absolute_error)
        17.66s   = Training   runtime
        0.1s     = Validation runtime
Fitting model: CatBoost_r177_BAG_L1 ... Training model for up to 701.32s of the 1379.58s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 16.81% memory usage per fold,
67.22%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=16.81%)
        -10.9312         = Validation score   (-mean_absolute_error)
        280.51s  = Training   runtime
        0.06s    = Validation runtime
Fitting model: NeuralNetTorch_r79_BAG_L1 ... Training model for up to 398.32s of the 1076.58s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.03%)
        -10.84   = Validation score   (-mean_absolute_error)
        73.33s   = Training   runtime
        0.18s    = Validation runtime
Fitting model: LightGBM_r131_BAG_L1 ... Training model for up to 304.41s of the 982.67s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.45%)
        -10.799  = Validation score   (-mean_absolute_error)
        8.69s    = Training   runtime
        0.25s    = Validation runtime
Fitting model: NeuralNetFastAI_r191_BAG_L1 ... Training model for up to 274.31s of the 952.57s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.07%)
        -11.6338         = Validation score   (-mean_absolute_error)
        40.42s   = Training   runtime
        0.36s    = Validation runtime
Fitting model: CatBoost_r9_BAG_L1 ... Training model for up to 211.63s of the 889.89s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 2 folds in parallel instead (Estimated 38.15% memory usage per fold,
76.30%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (2 workers, per: cpus=4, gpus=0,
memory=38.15%)
        -10.9141         = Validation score   (-mean_absolute_error)
        195.56s  = Training   runtime
        0.06s    = Validation runtime
Fitting model: LightGBM_r96_BAG_L1 ... Training model for up to 4.30s of the 682.56s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.15%)
        -11.2264         = Validation score   (-mean_absolute_error)
        6.81s    = Training   runtime
        0.23s    = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 360.00s of the 656.12s of remaining time.
        Ensemble Weights: {'LightGBMXT_BAG_L1': 0.522, 'NeuralNetTorch_r79_BAG_L1': 0.348, 'ExtraTreesMSE_BAG_L1': 0.13}
        -10.2182         = Validation score   (-mean_absolute_error)
        0.23s    = Training   runtime
        0.0s     = Validation runtime
Fitting 106 L2 models, fit_strategy="sequential" ...
Fitting model: LightGBMXT_BAG_L2 ... Training model for up to 655.83s of the 655.62s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.34%)
        -10.6004         = Validation score   (-mean_absolute_error)
        23.04s   = Training   runtime
        0.26s    = Validation runtime
Fitting model: LightGBM_BAG_L2 ... Training model for up to 613.53s of the 613.33s of remaining time.
```

```
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.34%)
        -10.8584         = Validation score   (-mean_absolute_error)
        20.67s   = Training   runtime
        0.14s    = Validation runtime
Fitting model: RandomForestMSE_BAG_L2 ... Training model for up to 573.13s of the 572.93s of remaining time.
        -10.7971         = Validation score   (-mean_absolute_error)
        2.21s    = Training   runtime
        0.36s    = Validation runtime
Fitting model: CatBoost_BAG_L2 ... Training model for up to 570.41s of the 570.21s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 12.83% memory usage per fold,
51.33%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=12.83%)
        -10.4622         = Validation score   (-mean_absolute_error)
        96.38s   = Training   runtime
        0.09s    = Validation runtime
Fitting model: ExtraTreesMSE_BAG_L2 ... Training model for up to 461.11s of the 460.90s of remaining time.
        -10.6112         = Validation score   (-mean_absolute_error)
        1.33s    = Training   runtime
        0.37s    = Validation runtime
Fitting model: NeuralNetFastAI_BAG_L2 ... Training model for up to 459.26s of the 459.05s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.05%)
        -11.033  = Validation score   (-mean_absolute_error)
        25.02s   = Training   runtime
        0.32s    = Validation runtime
Fitting model: XGBoost_BAG_L2 ... Training model for up to 417.12s of the 416.92s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=1.69%)
        -11.0744         = Validation score   (-mean_absolute_error)
        7.92s    = Training   runtime
        0.15s    = Validation runtime
Fitting model: NeuralNetTorch_BAG_L2 ... Training model for up to 387.19s of the 386.98s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.05%)
        -10.6148         = Validation score   (-mean_absolute_error)
        46.74s   = Training   runtime
        0.25s    = Validation runtime
Fitting model: LightGBMLarge_BAG_L2 ... Training model for up to 322.05s of the 321.84s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=1.23%)
        -11.3561         = Validation score   (-mean_absolute_error)
        12.89s   = Training   runtime
        0.1s     = Validation runtime
Fitting model: CatBoost_r177_BAG_L2 ... Training model for up to 287.97s of the 287.77s of remaining time.
        Memory not enough to fit 8 folds in parallel. Will train 4 folds in parallel instead (Estimated 13.99% memory usage per fold,
55.98%/80.00% total).
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (4 workers, per: cpus=2, gpus=0,
memory=13.99%)
        -10.4736         = Validation score   (-mean_absolute_error)
        98.32s   = Training   runtime
        0.06s    = Validation runtime
Fitting model: NeuralNetTorch_r79_BAG_L2 ... Training model for up to 178.70s of the 178.50s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.03%)
        -10.4154         = Validation score   (-mean_absolute_error)
        49.31s   = Training   runtime
        0.23s    = Validation runtime
Fitting model: LightGBM_r131_BAG_L2 ... Training model for up to 108.85s of the 108.64s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.55%)
        -10.9376         = Validation score   (-mean_absolute_error)
        37.57s   = Training   runtime
        0.48s    = Validation runtime
Fitting model: NeuralNetFastAI_r191_BAG_L2 ... Training model for up to 50.03s of the 49.82s of remaining time.
        Fitting 8 child models (S1F1 - S1F8) | Fitting with ParallelLocalFoldFittingStrategy (8 workers, per: cpus=1, gpus=0,
memory=0.05%)
        -10.7539         = Validation score   (-mean_absolute_error)
        33.27s   = Training   runtime
        0.54s    = Validation runtime
Fitting model: WeightedEnsemble_L3 ... Training model for up to 360.00s of the -3.19s of remaining time.
        Ensemble Weights: {'NeuralNetTorch_r79_BAG_L2': 0.476, 'LightGBMXT_BAG_L1': 0.381, 'NeuralNetFastAI_r191_BAG_L2': 0.095,
'CatBoost_BAG_L2': 0.048}
        -10.1579         = Validation score   (-mean_absolute_error)
        0.14s    = Training   runtime
        0.0s     = Validation runtime
AutoGluon training complete, total runtime = 2037.46s ... Best model: WeightedEnsemble_L3 | Estimated inference throughput: 46.5 rows/s
(93 batch size)
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("C:\Users\EIJEL\OneDrive - Bayer\Personal
Data\Erik\2024\pkl_new\cotton-dormancy\notebooks\cotton_initial_germ\AutogluonModels_20251223_124744")
```

In [82]:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
# Predict on test (continuous scores)
test_preds = predictor.predict(test_split.drop('CT_Current', axis=1), as_pandas=True)
y_true = test_split['CT_Current']  # True scores

# Evaluate regression metrics
print(f"Mean Absolute Error (MAE): {mean_absolute_error(y_true, test_preds):.2f}")
print(f"Root Mean Squared Error (RMSE): {mean_squared_error(y_true, test_preds):.2f}")  # Fixed: Use root_mean_
print(f"R² Score: {r2_score(y_true, test_preds):.4f}")

# Leaderboard on test
print(predictor.leaderboard(test_split, silent=True))  # Features only for leaderboard
```

```
Mean Absolute Error (MAE): 10.33
Root Mean Squared Error (RMSE): 188.09
R² Score: 0.7266
                      model  score_test  score_val          eval_metric  \
0               XGBoost_BAG_L1  -10.110251 -11.022360  mean_absolute_error
1        RandomForestMSE_BAG_L2  -10.182523 -10.797059  mean_absolute_error
2          LightGBM_r131_BAG_L1  -10.211213 -10.798963  mean_absolute_error
3          LightGBMLarge_BAG_L1  -10.237397 -11.183672  mean_absolute_error
4             LightGBMXT_BAG_L2  -10.263155 -10.600425  mean_absolute_error
5           WeightedEnsemble_L3  -10.329546 -10.157939  mean_absolute_error
```

6	LightGBM_r131_BAG_L2	-10.345030	-10.937602	mean_absolute_error
7	LightGBMXT_BAG_L1	-10.345063	-10.483300	mean_absolute_error
8	CatBoost_r177_BAG_L2	-10.381860	-10.473588	mean_absolute_error
9	ExtraTreesMSE_BAG_L2	-10.419416	-10.611246	mean_absolute_error
10	NeuralNetFastAI_r191_BAG_L2	-10.421611	-10.753859	mean_absolute_error
11	LightGBM_BAG_L1	-10.437282	-10.713856	mean_absolute_error
12	NeuralNetFastAI_BAG_L2	-10.453492	-11.032971	mean_absolute_error
13	LightGBM_BAG_L2	-10.472841	-10.858397	mean_absolute_error
14	CatBoost_BAG_L2	-10.514825	-10.462210	mean_absolute_error
15	WeightedEnsemble_L2	-10.531338	-10.218227	mean_absolute_error
16	XGBoost_BAG_L2	-10.545014	-11.074399	mean_absolute_error
17	RandomForestMSE_BAG_L1	-10.596726	-11.036887	mean_absolute_error
18	ExtraTreesMSE_BAG_L1	-10.603136	-10.729961	mean_absolute_error
19	NeuralNetTorch_r79_BAG_L2	-10.612096	-10.415424	mean_absolute_error
20	LightGBMLarge_BAG_L2	-10.695667	-11.356066	mean_absolute_error
21	CatBoost_BAG_L1	-10.700701	-10.780738	mean_absolute_error
22	NeuralNetTorch_BAG_L2	-10.785801	-10.614835	mean_absolute_error
23	CatBoost_r177_BAG_L1	-10.807959	-10.931157	mean_absolute_error
24	CatBoost_r9_BAG_L1	-10.947331	-10.914069	mean_absolute_error
25	NeuralNetTorch_r79_BAG_L1	-11.307744	-10.840032	mean_absolute_error
26	NeuralNetTorch_BAG_L1	-11.334094	-10.964384	mean_absolute_error
27	LightGBM_r96_BAG_L1	-11.529685	-11.226360	mean_absolute_error
28	NeuralNetFastAI_r191_BAG_L1	-11.744208	-11.633819	mean_absolute_error
29	NeuralNetFastAI_BAG_L1	-11.826849	-11.605833	mean_absolute_error

	pred_time_test	pred_time_val	fit_time	pred_time_test_marginal	\
0	0.465939	0.151282	12.477468	0.465939	
1	2.716152	1.936076	469.917230	0.212778	
2	0.534437	0.248867	8.691906	0.534437	
3	0.348903	0.102769	17.663902	0.348903	
4	3.093879	1.843518	490.753623	0.590505	
5	3.163875	2.445339	646.810826	0.001756	
6	3.629855	2.056853	505.277707	1.126481	
7	0.674133	0.314040	18.870590	0.674133	
8	2.601743	1.642202	566.027265	0.098368	
9	2.749862	1.947973	469.039320	0.246488	
10	2.802377	2.119557	500.974158	0.299003	
11	0.306148	0.096649	6.625504	0.306148	
12	2.782621	1.894490	492.726444	0.279247	
13	2.805222	1.723674	488.375817	0.301847	
14	2.595324	1.673163	564.093346	0.091949	
15	1.190897	0.696773	93.117238	0.002939	
16	2.899194	1.728288	475.626790	0.395820	
17	0.325466	0.310029	1.889074	0.325466	
18	0.249580	0.194817	0.694834	0.249580	
19	2.771166	1.807781	517.018105	0.267792	
20	2.757271	1.679046	480.597078	0.253897	
21	0.240223	0.064285	330.365747	0.240223	
22	2.736592	1.830628	514.452731	0.233218	
23	0.129139	0.061137	280.514312	0.129139	
24	0.155512	0.060893	195.557126	0.155512	
25	0.264244	0.183355	73.325732	0.264244	
26	0.231177	0.281551	94.978074	0.231177	
27	0.395674	0.228152	6.810023	0.395674	
28	0.568494	0.363721	40.419879	0.568494	
29	0.283788	0.361282	30.085432	0.283788	

	pred_time_val_marginal	fit_time_marginal	stack_level	can_infer	\
0	0.151282	12.477468	1	True	
1	0.356988	2.208353	2	True	
2	0.248867	8.691906	1	True	
3	0.102769	17.663902	1	True	
4	0.264430	23.044746	2	True	
5	0.003014	0.142971	3	True	
6	0.477764	37.568830	2	True	
7	0.314040	18.870590	1	True	
8	0.063113	98.318388	2	True	
9	0.368884	1.330444	2	True	
10	0.540469	33.265281	2	True	
11	0.096649	6.625504	1	True	
12	0.315402	25.017567	2	True	
13	0.144585	20.666940	2	True	
14	0.094075	96.384469	2	True	
15	0.004562	0.226082	2	True	
16	0.149199	7.917913	2	True	
17	0.310029	1.889074	1	True	
18	0.194817	0.694834	1	True	
19	0.228692	49.309228	2	True	
20	0.099957	12.888201	2	True	
21	0.064285	330.365747	1	True	
22	0.251539	46.743855	2	True	
23	0.061137	280.514312	1	True	
24	0.060893	195.557126	1	True	
25	0.183355	73.325732	1	True	
26	0.281551	94.978074	1	True	
27	0.228152	6.810023	1	True	
28	0.363721	40.419879	1	True	
29	0.361282	30.085432	1	True	

	fit_order
0	7
1	19
2	12
3	9
4	17
5	30
6	28
7	1
8	26
9	21
10	29
11	2
12	22
13	18
14	20
15	16
16	23
17	3
18	5
19	27
20	25
21	4
22	24

```
23        10
24        14
25        11
26         8
27        15
28        13
29         6
```

In [ ]:
```
# predictor = TabularPredictor.load('C:/tmp/autogluon_models_20251109_132945')
predictor_class = TabularPredictor.load(r'C:\Users\EIJEL\Documents\tosend\notebooks\cotton_initial_germ\Autogl
# Kayode look in the current dir of this notebook
# Predict: predictor.predict(new_df, model='LightGBMXT_BAG_L2')
```

In [ ]:
```
##################################################### check diff thresholds ##################################
```

In [ ]:
```
test_probs = predictor_class.predict_proba(test_split.drop('label', axis=1))
from sklearn.metrics import recall_score
for thresh in [0.2, 0.3, 0.4, 0.5]:
    preds_thresh = (test_probs[1] > thresh).astype(int)
    print(f"Thresh {thresh}: Poor Recall = {recall_score(y_true, preds_thresh)}")
```

In [ ]:
```
test_probs = predictor.predict_proba(test_split.drop('label', axis=1), model='NeuralNetFastAI_r102_BAG_L1')
from sklearn.metrics import recall_score
for thresh in [0.2, 0.3, 0.4, 0.5]:
    preds_thresh = (test_probs[1] > thresh).astype(int)
    print(f"Thresh {thresh}: Poor Recall = {recall_score(y_true, preds_thresh)}")
```

In [ ]:
```
test_split.head(2)
```

In [ ]:
```
##################################################### predict on new data ####################################
```

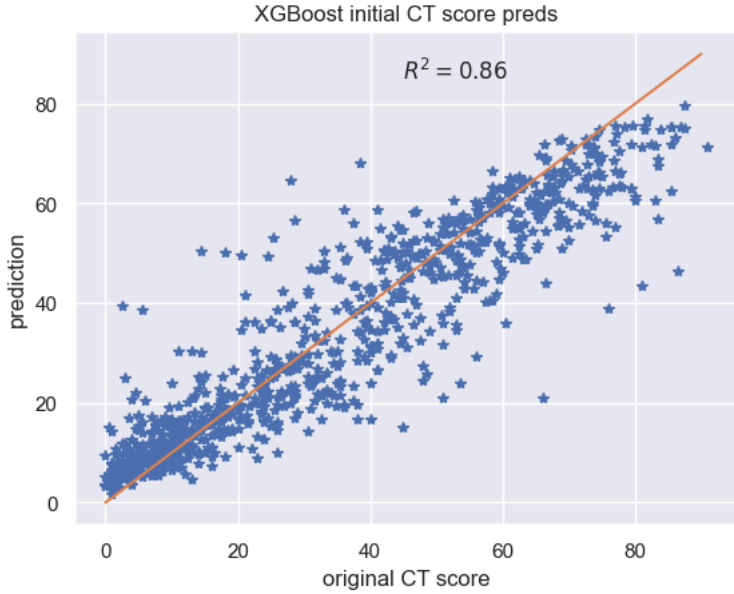
In [ ]:
```
### get residuals and cm ####
```

In [28]:
```
import pandas as pd
from autogluon.tabular import TabularPredictor
from sklearn.metrics import confusion_matrix  # For eval if you have labels

# Load (replace with your path if needed)
predictor_class = TabularPredictor.load(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cott
predictor_reg = TabularPredictor.load(r"C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton
```

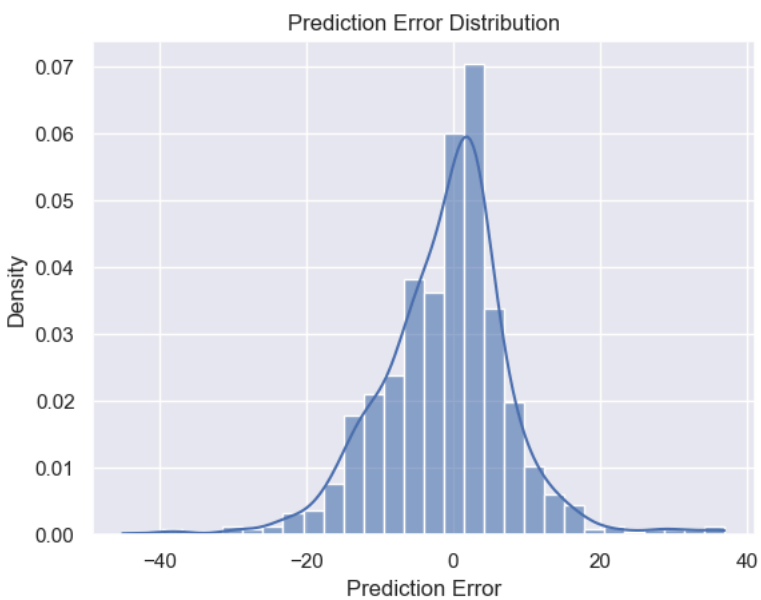
```
C:\Users\EIJEL\AppData\Local\Programs\Python\Python311\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please
update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

In [29]:
```
preds = predictor_reg.predict(dftotrain)
```

In [51]:
```
y_true = dftotrain.CT_Current
r2 = r2_score(y_true, preds)

plt.plot(dftotrain.CT_Current, preds, '*')
plt.plot([0, 90], [0, 90])
plt.xlabel('original CT score')
plt.ylabel('prediction')
plt.title('XGBoost initial CT score preds')
plt.text(45, 85, rf"$R^2 = {r2:.2}$")
plt.savefig("residuals1223feedbackr2.png", dpi=600)
```



In [48]:
```
# plt.hist(preds-dftotrain.CT_Current)
errors = preds - dftotrain["CT_Current"]
sns.histplot(
    errors,
    bins=30,
    kde=True,
    stat="density",   # important so KDE scales correctly
    # color="steelblue",
    alpha=0.6
)

plt.xlabel("Prediction Error")
plt.ylabel("Density")
plt.title("Prediction Error Distribution")
plt.savefig("residuals1223feedbackhist.png", dpi=600)
# plt.show()
```



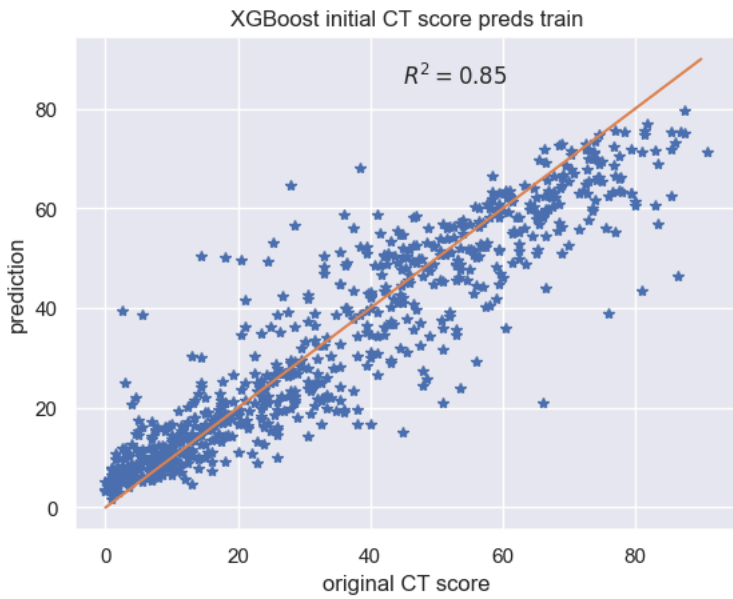
In [ ]:
```
# examine train and test in dftotrain for ppt
```

In [32]:
```
preds_train_split = predictor_reg.predict(train_split)
```

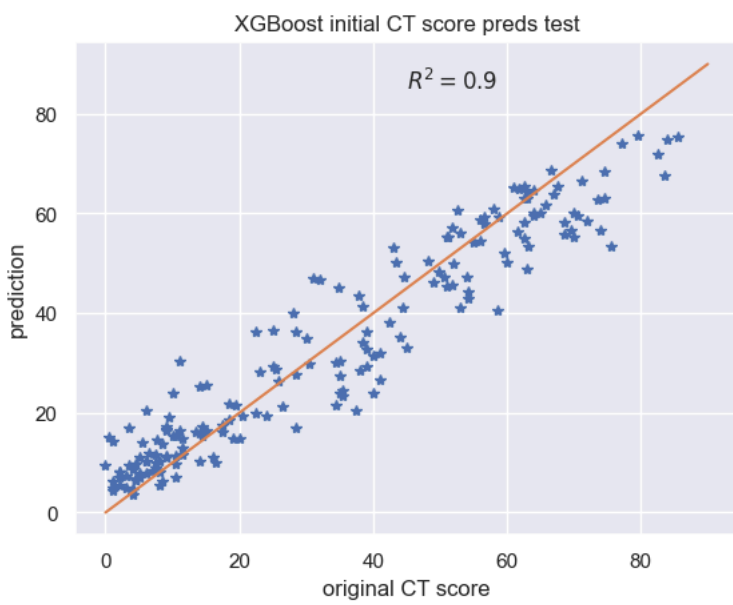
In [33]:
```
preds_test_split = predictor_reg.predict(test_split)
```

In [52]:
```
from sklearn.metrics import r2_score

y_true = dftotrain.loc[train_split.index].CT_Current
r2 = r2_score(y_true, preds_train_split)
```

```
plt.plot(dftotrain.loc[train_split.index].CT_Current, preds_train_split, '*')
plt.plot([0, 90], [0, 90])
plt.xlabel('original CT score')
plt.ylabel('prediction')
plt.text(45, 85, rf"$R^2 = {r2:.2}$")
plt.title('XGBoost initial CT score preds train')
plt.savefig("residuals1223feedback_train.png", dpi=600)
```



In [53]:
```
y_true = dftotrain.loc[test_split.index].CT_Current
r2 = r2_score(y_true, preds_test_split)

plt.plot(dftotrain.loc[test_split.index].CT_Current, preds_test_split, '*')
plt.plot([0, 90], [0, 90])
plt.xlabel('original CT score')
plt.ylabel('prediction')
plt.text(45, 85, rf"$R^2 = {r2:.2}$")
plt.title('XGBoost initial CT score preds test')
plt.savefig("residuals1223feedback_test.png", dpi=600)
```



In [58]:
```
predictor_reg.info()["model_info"]["WeightedEnsemble_L3"]
```

Out [58]:
```
{'name': 'WeightedEnsemble_L3',
 'model_type': 'WeightedEnsembleModel',
 'problem_type': 'regression',
 'eval_metric': 'mean_absolute_error',
 'stopping_metric': 'mean_absolute_error',
 'fit_time': 0.14297056198120117,
 'num_classes': None,
```

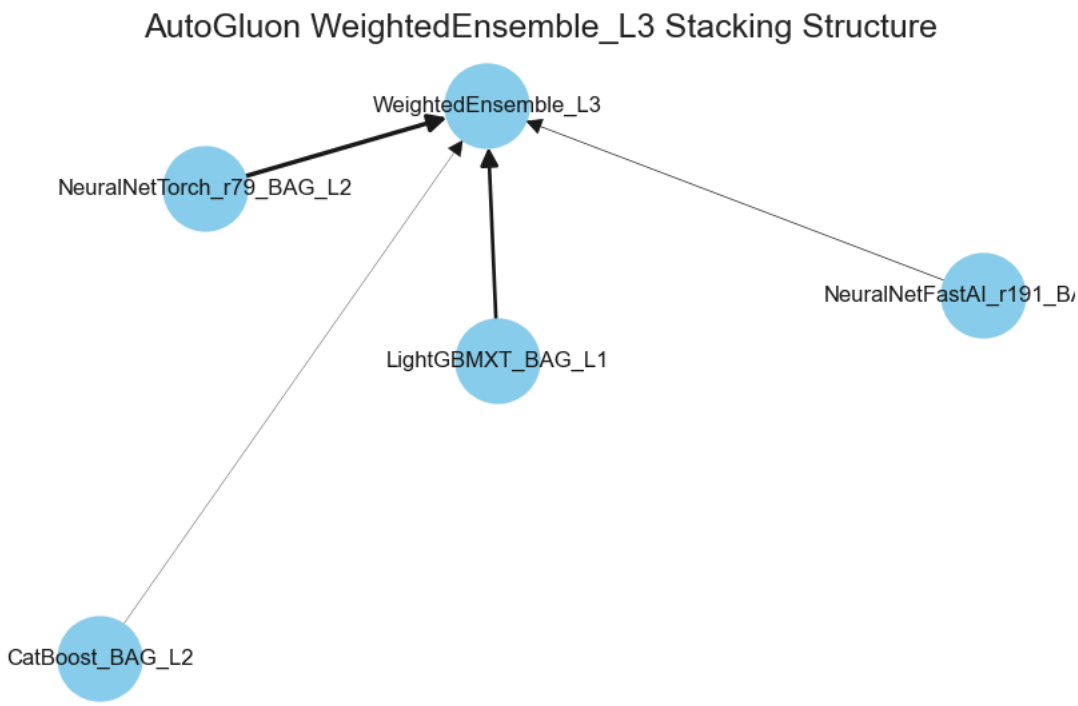
```
 'quantile_levels': None,
 'predict_time': 0.0030143260955810547,
 'val_score': -10.15793916565996,
 'hyperparameters': {'use_orig_features': False,
  'valid_stacker': True,
  'max_base_models': 0,
  'max_base_models_per_type': 'auto',
  'save_bag_folds': True,
  'stratify': 'auto',
  'bin': 'auto',
  'n_bins': None},
 'hyperparameters_user': {'save_bag_folds': True},
 'hyperparameters_fit': {},
 'hyperparameters_nondefault': ['save_bag_folds'],
 'ag_args_fit': {'max_memory_usage_ratio': 1.0,
  'max_time_limit_ratio': 1.0,
  'max_time_limit': None,
  'min_time_limit': 0,
  'valid_raw_types': None,
  'valid_special_types': None,
  'ignored_type_group_special': None,
  'ignored_type_group_raw': None,
  'get_features_kwargs': None,
  'get_features_kwargs_extra': None,
  'predict_1_batch_size': None,
  'temperature_scalar': None,
  'drop_unique': False},
 'num_features': 4,
 'features': ['NeuralNetTorch_r79_BAG_L2',
  'LightGBMXT_BAG_L1',
  'CatBoost_BAG_L2',
  'NeuralNetFastAI_r191_BAG_L2'],
 'feature_metadata': <autogluon.common.features.feature_metadata.FeatureMetadata at 0x2535d87df90>,
 'memory_size': 5386,
 'compile_time': None,
 'is_initialized': True,
 'is_fit': True,
 'is_valid': True,
 'can_infer': True,
 'has_learning_curves': False,
 'num_samples': 738,
 'val_in_fit': False,
 'unlabeled_in_fit': False,
 'num_cpus': 8,
 'num_gpus': 0,
 'bagged_info': {'child_model_type': 'GreedyWeightedEnsembleModel',
  'num_child_models': 1,
  'child_model_names': ['S1F1'],
  '_n_repeats': 1,
  '_k_per_n_repeat': [1],
  '_random_state': 3,
  'low_memory': False,
  'bagged_mode': False,
  'max_memory_size': 5386,
  'min_memory_size': 5386,
  'child_hyperparameters': {'ensemble_size': 25, 'subsample_size': 1000000},
  'child_hyperparameters_user': {},
  'child_hyperparameters_fit': {'ensemble_size': 21},
  'child_ag_args_fit': {'max_memory_usage_ratio': 1.0,
   'max_time_limit_ratio': 1.0,
   'max_time_limit': None,
   'min_time_limit': 0,
   'valid_raw_types': None,
   'valid_special_types': None,
   'ignored_type_group_special': None,
   'ignored_type_group_raw': None,
   'get_features_kwargs': None,
   'get_features_kwargs_extra': None,
   'predict_1_batch_size': None,
   'temperature_scalar': None,
   'drop_unique': False}},
 'stacker_info': {'num_base_models': 4,
  'base_model_names': ['LightGBMXT_BAG_L1',
   'CatBoost_BAG_L2',
   'NeuralNetTorch_r79_BAG_L2',
   'NeuralNetFastAI_r191_BAG_L2']},
 'children_info': {'S1F1': {'name': 'S1F1',
   'model_type': 'GreedyWeightedEnsembleModel',
   'problem_type': 'regression',
   'eval_metric': 'mean_absolute_error',
   'stopping_metric': 'mean_absolute_error',
   'fit_time': 0.14297056198120117,
   'num_classes': None,
   'quantile_levels': None,
   'predict_time': None,
   'val_score': None,
   'hyperparameters': {'ensemble_size': 25, 'subsample_size': 1000000},
   'hyperparameters_user': {},
   'hyperparameters_fit': {'ensemble_size': 21},
   'hyperparameters_nondefault': [],
   'ag_args_fit': {'max_memory_usage_ratio': 1.0,
    'max_time_limit_ratio': 1.0,
    'max_time_limit': None,
    'min_time_limit': 0,
    'valid_raw_types': None,
    'valid_special_types': None,
    'ignored_type_group_special': None,
    'ignored_type_group_raw': None,
    'get_features_kwargs': None,
    'get_features_kwargs_extra': None,
    'predict_1_batch_size': None,
    'temperature_scalar': None,
    'drop_unique': False},
   'num_features': 4,
   'features': ['LightGBMXT_BAG_L1',
    'CatBoost_BAG_L2',
    'NeuralNetTorch_r79_BAG_L2',
    'NeuralNetFastAI_r191_BAG_L2'],
   'feature_metadata': <autogluon.common.features.feature_metadata.FeatureMetadata at 0x2535e1c4d90>,
   'memory_size': 6362,
   'compile_time': None,
   'is_initialized': True,
   'is_fit': True,
```

```
  'is_valid': True,
  'can_infer': True,
  'has_learning_curves': False,
  'num_samples': 738,
  'val_in_fit': False,
  'unlabeled_in_fit': False,
  'num_cpus': 8,
  'num_gpus': 0,
  'model_weights': {'LightGBMXT_BAG_L1': 0.380952380952381,
   'CatBoost_BAG_L2': 0.04761904761904762,
   'NeuralNetTorch_r79_BAG_L2': 0.4761904761904762,
   'NeuralNetFastAI_r191_BAG_L2': 0.09523809523809525}}}}
```

In [62]:

```python
import matplotlib.pyplot as plt
import networkx as nx

# Models and weights
models = {
    'LightGBMXT_BAG_L1': 0.381,
    'CatBoost_BAG_L2': 0.048,
    'NeuralNetTorch_r79_BAG_L2': 0.476,
    'NeuralNetFastAI_r191_BAG_L2': 0.095
}

# Create graph
G = nx.DiGraph()

# Add nodes
G.add_node("WeightedEnsemble_L3")
for m in models:
    G.add_node(m)
    G.add_edge(m, "WeightedEnsemble_L3", weight=models[m])

# Plot
pos = nx.spring_layout(G, seed=42)
edges = G.edges()
weights = [G[u][v]['weight']*5 for u,v in edges]  # scale for visibility

plt.figure(figsize=(8,5))
nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=2000,
        arrowsize=20, width=weights)
plt.title("AutoGluon WeightedEnsemble_L3 Stacking Structure")
plt.savefig("weighted_ensemble_L3.png", dpi=600)
plt.show()

# import matplotlib.pyplot as plt
# import networkx as nx

# # Models and weights
# models = {
#     'LightGBMXT_BAG_L1': 0.381,
#     'CatBoost_BAG_L2': 0.048,
#     'NeuralNetTorch_r79_BAG_L2': 0.476,
#     'NeuralNetFastAI_r191_BAG_L2': 0.095
# }

# # Create directed graph
# G = nx.DiGraph()
# G.add_node("WeightedEnsemble_L3", layer="L3")

# # Add base models as nodes and connect to ensemble
# for m, w in models.items():
#     G.add_node(m, layer="L1/L2")
#     G.add_edge(m, "WeightedEnsemble_L3", weight=w)

# # Layout: manually separate layers for clarity
# pos = {
#     'LightGBMXT_BAG_L1': (-1, 1),
#     'CatBoost_BAG_L2': (-1, 0),
#     'NeuralNetTorch_r79_BAG_L2': (-1, -1),
#     'NeuralNetFastAI_r191_BAG_L2': (-1, -2),
#     'WeightedEnsemble_L3': (1, -0.5)
# }

# # Edge widths scaled by weight
# edges = G.edges()
# weights = [G[u][v]['weight']*8 for u,v in edges]  # scale for visibility
```

```
# plt.figure(figsize=(8,5))
# nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=2000,
#         arrowsize=20, width=weights)

# # Add edge labels (fractional contribution)
# edge_labels = {(u,v): f"{G[u][v]['weight']:.2f}" for u,v in edges}
# nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='red', font_size=10)

# plt.title("AutoGluon WeightedEnsemble_L3 Stacking Structure\nArrow labels = fractional contribution")
# plt.axis('off')
# plt.show()
```



In [64]:
```
predictor_class.info()['model_info']['WeightedEnsemble_L2']
```

Out [64]:
```
{'name': 'WeightedEnsemble_L2',
 'model_type': 'WeightedEnsembleModel',
 'problem_type': 'binary',
 'eval_metric': 'f1_macro',
 'stopping_metric': 'f1_macro',
 'fit_time': 0.8578054904937744,
 'num_classes': 2,
 'quantile_levels': None,
 'predict_time': 0.0,
 'val_score': 0.8590692755156002,
 'hyperparameters': {'use_orig_features': False,
  'valid_stacker': True,
  'max_base_models': 0,
  'max_base_models_per_type': 'auto',
  'save_bag_folds': True,
  'stratify': 'auto',
  'bin': 'auto',
  'n_bins': None},
 'hyperparameters_user': {'save_bag_folds': True},
 'hyperparameters_fit': {},
 'hyperparameters_nondefault': ['save_bag_folds'],
 'ag_args_fit': {'max_memory_usage_ratio': 1.0,
  'max_time_limit_ratio': 1.0,
  'max_time_limit': None,
  'min_time_limit': 0,
  'valid_raw_types': None,
  'valid_special_types': None,
  'ignored_type_group_special': None,
  'ignored_type_group_raw': None,
  'get_features_kwargs': None,
  'get_features_kwargs_extra': None,
  'predict_1_batch_size': None,
  'temperature_scalar': None,
  'drop_unique': False},
 'num_features': 1,
 'features': ['LightGBM_r130_BAG_L1'],
 'feature_metadata': <autogluon.common.features.feature_metadata.FeatureMetadata at 0x2535dc4e250>,
 'memory_size': 5975,
 'compile_time': None,
 'is_initialized': True,
 'is_fit': True,
 'is_valid': True,
 'can_infer': True,
 'has_learning_curves': False,
 'num_samples': 738,
 'val_in_fit': False,
 'unlabeled_in_fit': False,
 'num_cpus': 8,
 'num_gpus': 0,
```

```
 'bagged_info': {'child_model_type': 'GreedyWeightedEnsembleModel',
  'num_child_models': 1,
  'child_model_names': ['S1F1'],
  '_n_repeats': 1,
  '_k_per_n_repeat': [1],
  '_random_state': 2,
  'low_memory': False,
  'bagged_mode': False,
  'max_memory_size': 5975,
  'min_memory_size': 5975,
  'child_hyperparameters': {'ensemble_size': 25, 'subsample_size': 1000000},
  'child_hyperparameters_user': {},
  'child_hyperparameters_fit': {'ensemble_size': 1},
  'child_ag_args_fit': {'max_memory_usage_ratio': 1.0,
   'max_time_limit_ratio': 1.0,
   'max_time_limit': None,
   'min_time_limit': 0,
   'valid_raw_types': None,
   'valid_special_types': None,
   'ignored_type_group_special': None,
   'ignored_type_group_raw': None,
   'get_features_kwargs': None,
   'get_features_kwargs_extra': None,
   'predict_1_batch_size': None,
   'temperature_scalar': None,
   'drop_unique': False}},
 'stacker_info': {'num_base_models': 1,
  'base_model_names': ['LightGBM_r130_BAG_L1']},
 'children_info': {'S1F1': {'name': 'S1F1',
   'model_type': 'GreedyWeightedEnsembleModel',
   'problem_type': 'binary',
   'eval_metric': 'f1_macro',
   'stopping_metric': 'f1_macro',
   'fit_time': 0.8578054904937744,
   'num_classes': 2,
   'quantile_levels': None,
   'predict_time': None,
   'val_score': None,
   'hyperparameters': {'ensemble_size': 25, 'subsample_size': 1000000},
   'hyperparameters_user': {},
   'hyperparameters_fit': {'ensemble_size': 1},
   'hyperparameters_nondefault': [],
   'ag_args_fit': {'max_memory_usage_ratio': 1.0,
    'max_time_limit_ratio': 1.0,
    'max_time_limit': None,
    'min_time_limit': 0,
    'valid_raw_types': None,
    'valid_special_types': None,
    'ignored_type_group_special': None,
    'ignored_type_group_raw': None,
    'get_features_kwargs': None,
    'get_features_kwargs_extra': None,
    'predict_1_batch_size': None,
    'temperature_scalar': None,
    'drop_unique': False},
   'num_features': 1,
   'features': ['LightGBM_r130_BAG_L1'],
   'feature_metadata': <autogluon.common.features.feature_metadata.FeatureMetadata at 0x2535a8d3150>,
   'memory_size': 5547,
   'compile_time': None,
   'is_initialized': True,
   'is_fit': True,
   'is_valid': True,
   'can_infer': True,
   'has_learning_curves': False,
   'num_samples': 738,
   'val_in_fit': False,
   'unlabeled_in_fit': False,
   'num_cpus': 8,
   'num_gpus': 0,
   'model_weights': {'LightGBM_r130_BAG_L1': 1.0}}}}
```

In [86]:
```
new_probs = predictor_class.predict(dftotrain)
```

In [89]:
```
print(new_probs.shape)
print(cold_class.shape)
```

```
(923,)

---------------------------------------------------------------------------AttributeError                            Traceback (most recent call last)
      1 print(new_probs.shape)
----> 2 print(cold_class.shape)
AttributeError: 'dict' object has no attribute 'shape'
```

In [52]:
```

```

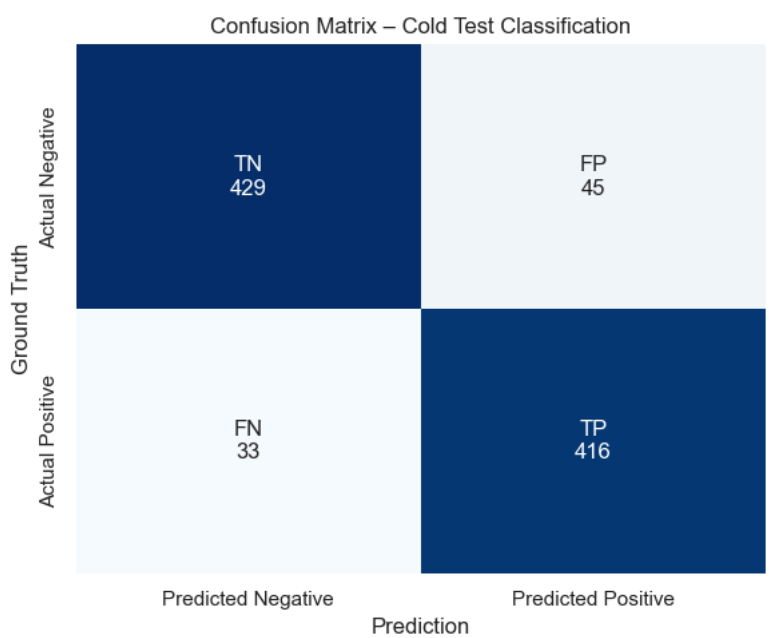
In [90]:
```
import numpy as np
from sklearn.metrics import confusion_matrix

cold_class = np.array((dftotrain['CT_Current'] < 30).astype(int))
cm = confusion_matrix(cold_class, new_probs)
```

In [91]:
```
cm
```

Out [91]:
```
array([[429,  45],
       [ 33, 416]], dtype=int64)
```

In [92]:
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# cm = confusion_matrix(y_true, y_pred)
# cm layout:
# [[TN, FP],
#  [FN, TP]]

tn, fp, fn, tp = cm.ravel()

labels = np.array([
    [f"TN\n{tn}", f"FP\n{fp}"],
    [f"FN\n{fn}", f"TP\n{tp}"]
])

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=labels,
    fmt="",
    cmap="Blues",
    cbar=False,
    xticklabels=["Predicted Negative", "Predicted Positive"],
    yticklabels=["Actual Negative", "Actual Positive"]
)

plt.xlabel("Prediction")
plt.ylabel("Ground Truth")
plt.title("Confusion Matrix – Cold Test Classification")
plt.tight_layout()
plt.savefig("cm_from_autogluon_1223feedback.png", dpi=600)
plt.show()
```



Feature importance

In [33]:
```
# feature importance
from sklearn.model_selection import train_test_split
train_data = dftotrain.copy()
train_data['label'] = (train_data['CT_Current'] < 30).astype(int)  # 1=poor, 0=good
# train_data = train_data.drop('CT_Current', axis=1)
train_split, test_split = train_test_split(
    train_data, test_size=0.2, stratify=train_data['label'], random_state=42
)
```

In [94]:
```
importance = predictor_reg.feature_importance(
    data=test_split,  # pandas DataFrame, required for accurate results
    feature_stage='original'    # Use 'original' for your raw input features
)

print(importance)
```

```
# This returns a DataFrame with columns like 'importance', 'stddev', 'p_value', etc.
# Sort by 'importance' descending for top features
print(importance.sort_values(by='importance', ascending=False))
```

```
These features in provided data are not utilized by the predictor and will be ignored: ['label']
Computing feature importance via permutation shuffling for 20 features using 185 rows with 5 shuffle sets...
        166.03s = Expected runtime (33.21s per shuffle set)
        39.55s  = Actual runtime (Completed 5 of 5 shuffle sets)

                              importance    stddev   p_value  n  p99_high  \
irrigation_type                 3.113393  0.562215  0.000122  5  4.271002
defol_to_harvest_average_dd60   2.763474  0.235274  0.000006  5  3.247907
year                            2.107034  0.177658  0.000006  5  2.472834
defol_to_harvest_average_heat   1.619312  0.104438  0.000002  5  1.834350
rm                              1.466032  0.186367  0.000031  5  1.849764
defol_to_harvest_cum_dd60       1.431130  0.181917  0.000031  5  1.805699
pp_day5_cum_dd60                1.330605  0.163619  0.000027  5  1.667499
maczone                         1.239063  0.265137  0.000237  5  1.784984
dd60                            0.971340  0.143106  0.000055  5  1.265998
cumulated_dd60                  0.937207  0.153014  0.000082  5  1.252264
pp_day5_avg_soilmoisture        0.865020  0.315284  0.001789  5  1.514194
defol_to_harvest_cum_heat       0.785965  0.130135  0.000087  5  1.053915
pp_day5_avg_dd60                0.554826  0.110941  0.000182  5  0.783255
pp_day10_cum_soilmoisture       0.521504  0.164254  0.001040  5  0.859706
pp_day10_cum_dd60               0.513297  0.035958  0.000003  5  0.587334
cumulated_soil_moisture         0.500276  0.086883  0.000105  5  0.679169
pp_day10_avg_dd60               0.429368  0.091497  0.000233  5  0.617761
pp_day10_avg_soilmoisture       0.394046  0.095805  0.000388  5  0.591310
defol_to_harvest                0.354565  0.030430  0.000006  5  0.417221
pp_day5_cum_soilmoisture        0.352417  0.057628  0.000083  5  0.471073

                               p99_low
irrigation_type                1.955783
defol_to_harvest_average_dd60  2.279040
year                           1.741235
defol_to_harvest_average_heat  1.404273
rm                             1.082301
defol_to_harvest_cum_dd60      1.056561
pp_day5_cum_dd60               0.993712
maczone                        0.693141
dd60                           0.676683
cumulated_dd60                 0.622150
pp_day5_avg_soilmoisture       0.215846
defol_to_harvest_cum_heat      0.518015
pp_day5_avg_dd60               0.326396
pp_day10_cum_soilmoisture      0.183303
pp_day10_cum_dd60              0.439259
cumulated_soil_moisture        0.321383
pp_day10_avg_dd60              0.240975
pp_day10_avg_soilmoisture      0.196782
defol_to_harvest               0.291910
pp_day5_cum_soilmoisture       0.233761
                              importance    stddev   p_value  n  p99_high  \
irrigation_type                 3.113393  0.562215  0.000122  5  4.271002
defol_to_harvest_average_dd60   2.763474  0.235274  0.000006  5  3.247907
year                            2.107034  0.177658  0.000006  5  2.472834
defol_to_harvest_average_heat   1.619312  0.104438  0.000002  5  1.834350
rm                              1.466032  0.186367  0.000031  5  1.849764
defol_to_harvest_cum_dd60       1.431130  0.181917  0.000031  5  1.805699
pp_day5_cum_dd60                1.330605  0.163619  0.000027  5  1.667499
maczone                         1.239063  0.265137  0.000237  5  1.784984
dd60                            0.971340  0.143106  0.000055  5  1.265998
cumulated_dd60                  0.937207  0.153014  0.000082  5  1.252264
pp_day5_avg_soilmoisture        0.865020  0.315284  0.001789  5  1.514194
defol_to_harvest_cum_heat       0.785965  0.130135  0.000087  5  1.053915
pp_day5_avg_dd60                0.554826  0.110941  0.000182  5  0.783255
pp_day10_cum_soilmoisture       0.521504  0.164254  0.001040  5  0.859706
pp_day10_cum_dd60               0.513297  0.035958  0.000003  5  0.587334
cumulated_soil_moisture         0.500276  0.086883  0.000105  5  0.679169
pp_day10_avg_dd60               0.429368  0.091497  0.000233  5  0.617761
pp_day10_avg_soilmoisture       0.394046  0.095805  0.000388  5  0.591310
defol_to_harvest                0.354565  0.030430  0.000006  5  0.417221
pp_day5_cum_soilmoisture        0.352417  0.057628  0.000083  5  0.471073

                               p99_low
irrigation_type                1.955783
defol_to_harvest_average_dd60  2.279040
year                           1.741235
defol_to_harvest_average_heat  1.404273
rm                             1.082301
defol_to_harvest_cum_dd60      1.056561
pp_day5_cum_dd60               0.993712
maczone                        0.693141
dd60                           0.676683
cumulated_dd60                 0.622150
pp_day5_avg_soilmoisture       0.215846
defol_to_harvest_cum_heat      0.518015
pp_day5_avg_dd60               0.326396
pp_day10_cum_soilmoisture      0.183303
pp_day10_cum_dd60              0.439259
cumulated_soil_moisture        0.321383
pp_day10_avg_dd60              0.240975
pp_day10_avg_soilmoisture      0.196782
defol_to_harvest               0.291910
pp_day5_cum_soilmoisture       0.233761
```

In [46]:

```
# X = raw features only (NO label)
X = test_split.drop(columns=["label"])
def ag_predict_class(X):
    return predictor_class.predict_proba(X)[:, 1]

import shap

X_bg = X.sample(100, random_state=42).reset_index(drop=True)
X_explain = X_bg.copy()
```

```
explainer = shap.Explainer(
    ag_predict_class,
    masker=shap.maskers.Independent(X_bg),
    algorithm="permutation"
)
```

In [48]:

```
feature_names = X.columns.tolist()

def ag_predict_numpy(X_np):
    X_df = pd.DataFrame(X_np, columns=feature_names)
    return predictor_class.predict_proba(X_df)[:, 1]
X_bg_np = X_bg.values.astype(object)  # keep objects safe
import shap

explainer = shap.KernelExplainer(
    ag_predict_numpy,
    X_bg_np
)
shap_values = explainer.shap_values(
    X_bg_np,
    nsamples=100
)
```

```
Provided model function fails when applied to the provided data set.

---------------------------------------------------------------------------TypeError                                 Traceback (most recent call last)
   3804 try:
-> 3805     return self._engine.get_loc(casted_key)
   3806 except KeyError as err:
File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()
File index.pyx:173, in pandas._libs.index.IndexEngine.get_loc()
TypeError: '(slice(None, None, None), 1)' is an invalid key
During handling of the above exception, another exception occurred:
InvalidIndexError                         Traceback (most recent call last)Cell In[48], line 9
      6 X_bg_np = X_bg.values.astype(object)  # keep objects safe
      7 import shap
----> 9 explainer = shap.KernelExplainer(
     10     ag_predict_numpy,
     11     X_bg_np
     12 )
     13 shap_values = explainer.shap_values(
     14     X_bg_np,
     15     nsamples=100
     16 )
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\shap\explainers\_kernel.py:97, in KernelExplainer.__init__(self, model, data, feature
     95 self.model = convert_to_model(model, keep_index=self.keep_index)
     96 self.data = convert_to_data(data, keep_index=self.keep_index)
---> 97 model_null = match_model_to_data(self.model, self.data)
     99 # enforce our current input type limitations
    100 if not isinstance(self.data, (DenseData, SparseData)):
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\shap\utils\_legacy.py:139, in match_model_to_data(model, data)
    137         out_val = model.f(data.convert_to_df())
    138     else:
--> 139         out_val = model.f(data.data)
    140 except Exception:
    141     print("Provided model function fails when applied to the provided data set.")
Cell In[48], line 5, in ag_predict_numpy(X_np)
      3 def ag_predict_numpy(X_np):
      4     X_df = pd.DataFrame(X_np, columns=feature_names)
----> 5     return predictor_class.predict_proba(X_df)[:, 1]
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\pandas\core\frame.py:4102, in DataFrame.__getitem__(self, key)
   4100 if self.columns.nlevels > 1:
   4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
   4103 if is_integer(indexer):
   4104     indexer = [indexer]
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\pandas\core\indexes\base.py:3817, in Index.get_loc(self, key)
   3812     raise KeyError(key) from err
   3813 except TypeError:
   3814     # If we have a listlike key, _check_indexing_error will raise
   3815     #  InvalidIndexError. Otherwise we fall through and re-raise
   3816     #  the TypeError.
-> 3817     self._check_indexing_error(key)
   3818     raise
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\pandas\core\indexes\base.py:6059, in Index._check_indexing_error(self, key)
   6055 def _check_indexing_error(self, key):
   6056     if not is_scalar(key):
   6057         # if key is not a scalar, directly raise an error (the code below
   6058         # would convert to numpy arrays and raise later any way) - GH29926
-> 6059         raise InvalidIndexError(key)
InvalidIndexError: (slice(None, None, None), 1)
```

In [47]:

```
shap_values = explainer(X_explain)
```

```
---------------------------------------------------------------------------TypeError                                 Traceback (most recent call last)
----> 1 shap_values = explainer(X_explain)
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\shap\explainers\_permutation.py:77, in PermutationExplainer.__call__(self, max_evals,
     74 def __call__(self, *args, max_evals=500, main_effects=False, error_bounds=False, batch_size="auto",
     75             outputs=None, silent=False):
     76     """Explain the output of the model on the given arguments."""
---> 77     return super().__call__(
     78         *args, max_evals=max_evals, main_effects=main_effects, error_bounds=error_bounds, batch_size=batch_size,
     79         outputs=outputs, silent=silent
     80     )
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\shap\explainers\_explainer.py:266, in Explainer.__call__(self, max_evals, main_effect
    264     feature_names = [[] for _ in range(len(args))]
    265 for row_args in show_progress(zip(*args), num_rows, self.__class__.__name__+" explainer", silent):
```

```
--> 266     row_result = self.explain_row(
    267         *row_args, max_evals=max_evals, main_effects=main_effects, error_bounds=error_bounds,
    268         batch_size=batch_size, outputs=outputs, silent=silent, **kwargs
    269     )
    270     values.append(row_result.get("values", None))
    271     output_indices.append(row_result.get("output_indices", None))
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\shap\explainers\_permutation.py:85, in PermutationExplainer.explain_row(self, max_eva
     83 """Explains a single row and returns the tuple (row_values, row_expected_values, row_mask_shapes)."""
     84 # build a masked version of the model for the current input sample
---> 85 fm = MaskedModel(self.model, self.masker, self.link, self.linearize_link, *row_args)
     87 # by default we run 10 permutations forward and backward
     88 if max_evals == "auto":
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\shap\utils\_masked_model.py:31, in MaskedModel.__init__(self, model, masker, link, li
     29 # if the masker supports it, save what positions vary from the background
     30 if callable(getattr(self.masker, "invariants", None)):
---> 31     self._variants = ~self.masker.invariants(*args)
     32     self._variants_column_sums = self._variants.sum(0)
     33     self._variants_row_inds = [
     34         self._variants[:,i] for i in range(self._variants.shape[1])
     35     ]
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\shap\maskers\_tabular.py:150, in Tabular.invariants(self, x)
    144 if x.shape != self.data.shape[1:]:
    145     raise DimensionError(
    146         "The passed data does not match the background shape expected by the masker! The data of shape " + \
    147         str(x.shape) + " was passed while the masker expected data of shape " + str(self.data.shape[1:]) + "."
    148     )
--> 150 return np.isclose(x, self.data)
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\numpy\core\numeric.py:2348, in isclose(a, b, rtol, atol, equal_nan)
   2345     dt = multiarray.result_type(y, 1.)
   2346     y = asanyarray(y, dtype=dt)
-> 2348 xfin = isfinite(x)
   2349 yfin = isfinite(y)
   2350 if all(xfin) and all(yfin):
TypeError: ufunc 'isfinite' not supported for the input types, and the inputs could not be safely coerced to any supported types according to the cast
```

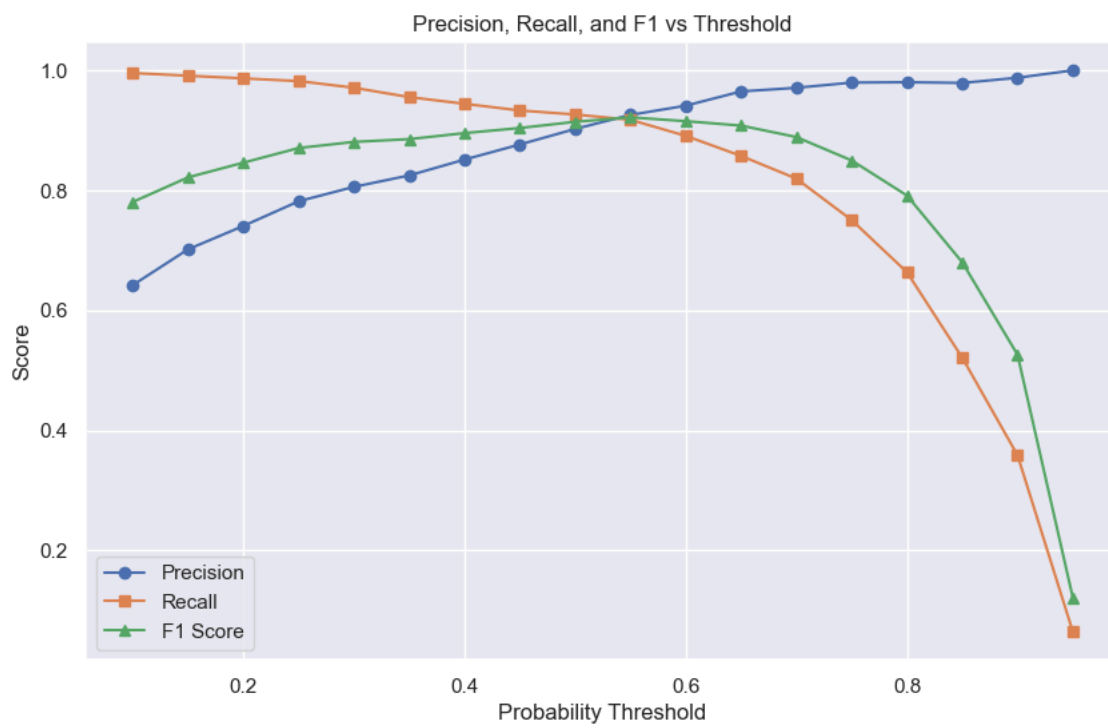
examine diff thresholds

In [100]:

```python
import pandas as pd
from sklearn.metrics import precision_score, recall_score, f1_score
from autogluon.tabular import TabularPredictor
import numpy as np
import matplotlib.pyplot as plt

# 1. Load your trained predictor
predictor = TabularPredictor.load(r"C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-dorr

# 2. Prepare your test/validation data (must have the true labels)
# Replace this with your actual test dataframe
# Example: test_data = pd.read_csv('your_test.csv')
# It needs the label column named 'label' (same as during training)
test_data = train_data.copy()

# 3. Get probability predictions for the positive class (class 1 = 'good' or whatever your positive is)
y_true = test_data['label'].values
proba_pos = predictor.predict_proba(test_data, model='WeightedEnsemble_L2')[1].values  # column 1 = positive cl

# 4. Try different thresholds and compute metrics
thresholds = np.arange(0.1, 1.0, 0.05)  # from 0.1 to 0.95 step 0.05 — adjust as needed

results = []

print(f"{'Threshold':>10} {'Precision':>12} {'Recall':>10} {'F1':>10} {'Support Pos':>12}")
print("-" * 60)

for thresh in thresholds:
    y_pred = (proba_pos >= thresh).astype(int)

    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    support_pos = y_pred.sum()  # number classified as positive

    results.append({
        'threshold': thresh,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'positive_predictions': support_pos
    })

    print(f"{thresh:10.2f} {precision:12.3f} {recall:10.3f} {f1:10.3f} {support_pos:12}")

# 5. Convert to DataFrame for easy viewing/saving
```

```
results_df = pd.DataFrame(results)
print("\nBest F1 threshold:")
print(results_df.loc[results_df['f1'].idxmax()])

# Optional: Plot the trade-off
plt.figure(figsize=(10, 6))
plt.plot(thresholds, [r['precision'] for r in results], label='Precision', marker='o')
plt.plot(thresholds, [r['recall'] for r in results], label='Recall', marker='s')
plt.plot(thresholds, [r['f1'] for r in results], label='F1 Score', marker='^')
plt.xlabel('Probability Threshold')
plt.ylabel('Score')
plt.title('Precision, Recall, and F1 vs Threshold')
plt.legend()
plt.grid(True)
plt.show()
```

```
 Threshold    Precision     Recall         F1  Support Pos
-----------------------------------------------------------
      0.10        0.642      0.996      0.781          696
      0.15        0.702      0.991      0.822          634
      0.20        0.741      0.987      0.846          598
      0.25        0.782      0.982      0.871          564
      0.30        0.806      0.971      0.881          541
      0.35        0.825      0.955      0.885          520
      0.40        0.851      0.944      0.895          498
      0.45        0.877      0.933      0.904          478
      0.50        0.902      0.927      0.914          461
      0.55        0.926      0.918      0.922          445
      0.60        0.941      0.891      0.915          425
      0.65        0.965      0.857      0.908          399
      0.70        0.971      0.820      0.889          379
      0.75        0.980      0.751      0.850          344
      0.80        0.980      0.664      0.792          304
      0.85        0.979      0.521      0.680          239
      0.90        0.988      0.359      0.526          163
      0.95        1.000      0.065      0.121           29

Best F1 threshold:
threshold                 0.550000
precision                 0.925843
recall                    0.917595
f1                        0.921700
positive_predictions    445.000000
Name: 9, dtype: float64
```



In [ ]:

In [ ]:

In [ ]:

```
import shap
import pandas as pd
import numpy as np
from autogluon.tabular import TabularPredictor
import fastai.tabular.all as fastai  # For raw learner

# Load predictor
```

```python
predictor = TabularPredictor.load('C:/tmp/autogluon_models_20251113_170119')
model_name = 'NeuralNetFastAI_r102_BAG_L1'

# Extract raw FastAI learner from first fold (bagged average)
model_path = predictor.path
fold_path = f"{model_path}/models/{model_name}/S1F1"  # First fold
learner = fastai.load_learner(fold_path)  # Loads the raw FastAI learner

# Prepare data (numeric only for SHAP; one-hot categoricals if needed)
X_explain = train_data.head(50).select_dtypes(include=[np.number])  # Numerics only
background = X_explain.head(10)
foreground = X_explain.head(20)

# Wrapper for FastAI learner.predict (returns (preds, _, _); take raw preds)
def fastai_predict(data):
    if isinstance(data, np.ndarray):
        data = pd.DataFrame(data, columns=background.columns)
    # FastAI predict on DataFrame
    dl = learner.dls.test_dl(data)
    preds, _, _ = learner.get_preds(dl=dl)
    return preds[:, 1].numpy().reshape(-1, 1)  # Prob class 1

# KernelExplainer
explainer = shap.KernelExplainer(fastai_predict, background)

# Compute (nsamples=50 for speed)
shap_values = explainer.shap_values(foreground, nsamples=50)

# Plots
import matplotlib.pyplot as plt
shap.summary_plot(shap_values, foreground, show=False)
plt.title(f'SHAP Summary for {model_name}')
plt.show()

shap.force_plot(explainer.expected_value, shap_values[0], foreground.iloc[0], show=False)
plt.title("SHAP Force Plot - Sample 0")
plt.show()

# Importance DF
shap_importance = np.abs(shap_values).mean(0)
shap_df = pd.DataFrame({'feature': foreground.columns, 'shap_importance': shap_importance.flatten()})
print("Top SHAP Features:")
print(shap_df.sort_values('shap_importance', ascending=False).head(10))
```

In [ ]:

In [ ]:

In [ ]: